

DISTANCE COVARIANCE-GUIDED ADAPTATION FOR HETEROGENEOUS EDGE WORKLOADS

by

ZAKI INDRA SUKMA

(Under the Direction of In Kee Kim)

ABSTRACT

Neural Radiance Fields (NeRFs) reconstruct 3D scenes from satellite imagery, but training them is computationally expensive—a problem on power-constrained edge devices such as the NVIDIA Jetson. On such devices, the hardware configuration (CPU/GPU frequency and active cores) strongly affects training throughput and power, yet efficient operating points are hard to find in a large, non-linear search space. This thesis first characterizes the configuration landscape of Sat-NeRF training through exhaustive profiling across four satellite scenes, establishing per-scene optimal configurations and showing that GPU frequency dominates performance, CPU cores are negligible, and energy efficiency peaks at an intermediate GPU frequency. It then applies an online optimizer that uses distance covariance to identify the influential hardware parameters and find near-optimal configurations within roughly ten iterations and an order of magnitude less device time than exhaustive search, while generalizing across scenes.

INDEX WORDS: Neural Radiance Fields, Sat-NeRF, Satellite imagery, 3D reconstruction, Edge computing, NVIDIA Jetson, DVFS, Energy efficiency, Hardware configuration optimization, Online optimization

DISTANCE COVARIANCE-GUIDED ADAPTATION FOR HETEROGENEOUS EDGE
WORKLOADS

by

ZAKI INDRA SUKMA

B.S., Universitas Gadjah Mada, 2017

A Thesis Submitted to the Graduate Faculty of the
University of Georgia in Partial Fulfillment of the Requirements for the Degree.

MASTER OF SCIENCE

ATHENS, GEORGIA

2026

©2026

Zaki Indra Sukma

All Rights Reserved

DISTANCE COVARIANCE-GUIDED ADAPTATION FOR HETEROGENEOUS EDGE
WORKLOADS

by

ZAKI INDRA SUKMA

Major Professor: In Kee Kim

Committee: Suchendra Bhandarkar
Lakshmish Ramaswamy
Deepak Mishra

Electronic Version Approved:

Anne Shaffer

Dean of the Graduate School

The University of Georgia

September 2026

DEDICATION

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

THIS PAGE IS OPTIONAL

ACKNOWLEDGMENTS

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

THIS PAGE IS OPTIONAL

CONTENTS

Acknowledgments	v
List of Figures	vii
List of Tables	x
1 Introduction	1
1.1 Motivation	1
1.2 Objective	3
1.3 Contributions	3
1.4 Structuring of Thesis	4
2 Background	5
2.1 Satellite 3D Reconstruction	5
2.2 Neural Radiance Fields as a Training Workload	6
2.3 Edge Computing and the NVIDIA Jetson Platform	7
2.4 Dynamic Voltage and Frequency Scaling and the Configuration Space	8
2.5 Hardware-Configuration Optimization	9
3 Methodology	11
3.1 Problem Formulation	11
3.2 Distance Covariance	12
3.3 Overview of the Optimizer	13
3.4 Efficiency and Feasibility	14

3.5	Correlation Analysis	14
3.6	Configuration Search	17
4	Evaluation	19
4.1	Experimental Setup	19
4.2	Hardware Parameter Characterization	22
4.3	ORACLE Configurations	37
4.4	Thermal and Accuracy Characteristics	39
4.5	Optimizer Validation	45
4.6	Search Space Reduction	49
4.7	Summary	50
5	Discussion and Related Works	51
5.1	Interpreting the Configuration Landscape	51
5.2	On the Behavior of the Optimizer	52
5.3	Practical Implications	53
5.4	Limitations	54
5.5	Related Work	55
6	Conclusion and Future Work	58
6.1	Summary and Contributions	58
6.2	Future Work	59
6.3	Closing Remarks	61
	Bibliography	62

LIST OF FIGURES

2.1	Overview of NeRF and its satellite variants. NeRF (a) uses a multilayer perceptron as its backbone. Shadow-NeRF (b) extends it with sky-color estimation, solar visibility, and an optional solar-correction unit for satellite imagery. Sat-NeRF (c) unifies the two and further handles bundle-adjusted RPC camera parameters and transient objects.	7
3.1	The optimizer, exploration phase (steps 1–4). Starting from a maximum-performance reference configuration (step 1), the optimizer samples diverse configurations by Latin Hypercube Sampling (step 4) and scores each one (steps 2–3): a feasible configuration (meeting the throughput target) is scored by its efficiency, while an infeasible one is penalized and avoided. Exploration continues until enough configurations have been seen to estimate how each hardware parameter affects throughput and power, after which the search proceeds to the exploitation phase (Figure 3.2).	15

3.2	The optimizer, exploitation phase (steps 5–10), continuing from exploration (Figure 3.1). The optimizer anchors on the best-scoring configurations (step 5), estimates each hardware parameter’s influence on throughput and power (step 6), and proposes the next configuration (steps 7–8) by adjusting high-influence parameters more aggressively than low-influence ones, scaling each step by that parameter’s influence and pinning parameters with negligible influence (here, CPU cores) to their minimum to save power (step 8). The loop refines the influence estimates while the search improves and perturbs it when it stalls (step 9), and validates the winning configuration once the budget is spent (step 10).	16
4.1	Sat-NeRF: training throughput versus GPU frequency.	23
4.2	Sat-NeRF: training throughput versus CPU frequency.	24
4.3	Sat-NeRF: training throughput versus CPU core count.	25
4.4	S-NeRF: training throughput versus GPU frequency.	26
4.5	S-NeRF: training throughput versus CPU frequency.	27
4.6	S-NeRF: training throughput versus CPU core count.	28
4.7	Sat-NeRF: throughput versus power for all profiled configurations, colored by GPU frequency (ORACLE starred).	30
4.8	Sat-NeRF: energy efficiency (iter/s per watt) versus GPU frequency; efficiency peaks at an intermediate frequency.	31
4.9	Sat-NeRF: projected full-training energy (50,000 steps) versus GPU frequency; energy is minimized at the intermediate efficiency-peak frequency.	32
4.10	Sat-NeRF: Pareto frontier of the throughput–power trade-off (ORACLE starred).	33
4.11	S-NeRF: throughput versus power for all profiled configurations, colored by GPU frequency (ORACLE starred).	34
4.12	S-NeRF: energy efficiency (iter/s per watt) versus GPU frequency; efficiency peaks at an intermediate frequency.	35
4.13	S-NeRF: projected full-training energy (50,000 steps) versus GPU frequency; energy is minimized at the intermediate efficiency-peak frequency.	36
4.14	S-NeRF: Pareto frontier of the throughput–power trade-off (ORACLE starred).	37

4.15	Sat-NeRF: GPU temperature versus GPU frequency.	40
4.16	Sat-NeRF: GPU temperature versus CPU core count.	41
4.17	Sat-NeRF: GPU temperature versus power, colored by GPU frequency.	42
4.18	S-NeRF: GPU temperature versus GPU frequency.	43
4.19	S-NeRF: GPU temperature versus CPU core count.	44
4.20	S-NeRF: GPU temperature versus power, colored by GPU frequency.	45
4.21	Sat-NeRF: optimizer search trajectory overlaid on all profiled configurations, colored by phase (ORACLE starred).	47
4.22	S-NeRF: optimizer search trajectory overlaid on the profiling, colored by phase (ORACLE starred).	48

LIST OF TABLES

4.1	Hardware configuration space. All 600 combinations were profiled exhaustively for each scene.	21
4.2	Pearson correlation between each hardware parameter and throughput / power, per scene (Sat-NeRF). GPU frequency dominates; CPU cores have near-zero impact on throughput.	22
4.3	Pearson correlation between each hardware parameter and throughput / power, per scene (S-NeRF). GPU frequency dominates; CPU cores have near-zero impact on throughput.	23
4.4	Per-scene ranges of throughput and power, and the best (ORACLE) efficiency (Sat-NeRF, target 3.0 iter/s). The hardware performance envelope is nearly scene-invariant.	29
4.5	Per-scene ranges of throughput and power, and the best (ORACLE) efficiency (S-NeRF, target 1.0 iter/s). S-NeRF tops out below Sat-NeRF, so its feasible target is 1.0 iter/s and its best efficiency is the peak τ/p over the space.	29
4.6	ORACLE (highest-efficiency feasible) configuration per scene (Sat-NeRF, target 3.0 iter/s).	38
4.7	ORACLE (highest-efficiency feasible) configuration per scene (S-NeRF, target 1.0 iter/s).	38
4.8	Cross-scene transferability: efficiency rank (out of 600) of each scene’s ORACLE when evaluated on every scene (Sat-NeRF). Diagonal (native) entries are rank 1 by definition.	38

4.9	Cross-scene transferability: efficiency rank (out of 600) of each scene's ORACLE when evaluated on every scene (S-NeRF). Diagonal (native) entries are rank 1 by definition.	39
4.10	Per-scene efficiency peak, thermal risk (configs exceeding 65°C), and reconstruction accuracy (Sat-NeRF).	39
4.11	Per-scene efficiency peak, thermal risk (configs exceeding 65°C), and reconstruction accuracy (S-NeRF).	39
4.12	The optimizer's best configuration versus the ORACLE (Sat-NeRF, target 3.0 iter/s). Configurations are cores/CPU MHz/GPU MHz; the optimizer's efficiency and gap are those of the selected configuration evaluated in the profiling, so the gap is the true efficiency shortfall to the ORACLE, and rank is that configuration's efficiency rank against the profiled distribution.	46
4.13	The optimizer's best configuration versus the ORACLE (S-NeRF, target 1.0 iter/s). Columns as in Table 4.12; ranks are over the 560 configurations above the minimum CPU frequency. Because the S-NeRF efficiency landscape is flat near its peak, the efficiency gap is a more meaningful measure of quality than the rank.	47
4.14	Optimizer search efficiency per scene (Sat-NeRF). The optimizer evaluates $\sim 1\%$ of the space yet reaches the top few percent.	49
4.15	Optimizer search efficiency per scene (S-NeRF). The optimizer evaluates $\sim 1\%$ of the space yet reaches within a few percent of the ORACLE efficiency; best ranks are over the 560 configurations above the minimum CPU frequency.	49
4.16	Recommended reduced search space, derived from the profiling landscape. This yields 78 configurations, an 87% reduction from the full space.	49

CHAPTER 1

INTRODUCTION

1.1 Motivation

The falling cost of launches and sensing hardware has produced an abundance of high-resolution satellite imagery, fueling applications in environmental monitoring, urban planning, disaster response, and agriculture. A central task in exploiting this imagery is the three-dimensional (3D) reconstruction of the Earth’s surface: transforming overlapping 2D satellite views into digital surface models (DSMs) and novel RGB views that reveal terrain, vegetation, and man-made structures.

Neural Radiance Fields (NeRF) [9] have recently emerged as a powerful approach to this problem, learning a continuous volumetric scene representation from a sparse set of images. Satellite-specific adaptations such as Shadow-NeRF [2] and Sat-NeRF [7] extend NeRF with models of solar illumination and transient effects, producing high-quality reconstructions from multi-view satellite imagery. This quality, however, comes at a steep computational cost: training a NeRF is an iterative, GPU-intensive process that consumes substantial time and energy for each scene.

At the same time, deep learning is increasingly moving to the *edge*. Deploying computation on resource-constrained devices (drones, field stations, and small satellites) reduces latency, conserves bandwidth, and enables autonomy where connectivity to the cloud is intermittent or unavailable. Platforms such as the NVIDIA Jetson family bring GPU acceleration into a compact, low power envelope, making in-situ 3D reconstruction with Sat-NeRF an attractive possibility. Running such

a training workload on the edge, however, collides with the device’s tight power and thermal budgets: the same computation that finishes quickly in a data center must instead be squeezed into a few watts.

Edge platforms expose this tension through a large set of tunable hardware parameters. Dynamic Voltage and Frequency Scaling (DVFS) allows the CPU and GPU frequencies, the number of active cores, and the memory clock to be adjusted, and the choice among these settings has an outsized effect on efficiency. Configurations that deliver the *same* throughput can differ substantially in power consumption: a poor choice wastes energy or violates a throughput target, while a good one sustains the required performance at minimal power. Finding these efficient operating points is difficult. The configuration space is large, the relationship between settings and performance is non-linear, and the hardware parameters interact: raising GPU frequency helps only when the CPU can keep the GPU fed. As a result, the efficient region cannot be predicted from individual settings and is easily missed.

Existing approaches to this problem each fall short for edge NeRF training. Manufacturer power presets fix the hardware parameters to a handful of coarse modes that cover only a small fraction of the space and routinely miss efficient points. Profiling-based methods such as ALERT [18] achieve near-optimal results but require exhaustive offline profiling that must be repeated for every new model, device, or scene, consuming hours of measurement. Learning-based controllers, whether reinforcement learning [22] or constrained Bayesian optimization [19], adapt online but need extensive training or exploration to converge. Moreover, nearly all of this work targets *inference*, a short and repeatable workload, and optimizes a single metric. NeRF *training* is different: it is long, resource-intensive, and effectively run once per scene, so spending a large profiling budget to tune a single run is wasteful. To date, no work has characterized the hardware-configuration landscape of NeRF training on edge devices or provided a lightweight optimizer that discovers efficient configurations without offline profiling.

1.2 Objective

This thesis addresses that gap with two complementary objectives. The first is to *characterize* the hardware-configuration landscape of Sat-NeRF training on a Jetson edge device through exhaustive profiling: measuring throughput, power, energy, and thermal behavior across the full configuration space and across multiple satellite scenes, in order to establish per-scene optimal (ORACLE) configurations and to expose the structural relationships between hardware parameters and performance. The second is to *optimize* configuration selection online. We apply CORAL (*Covariance-Guided Resource Adaptive Learning*) [11], an online, covariance-guided optimizer, to hardware configuration during Sat-NeRF training, and evaluate whether it can recover near-optimal configurations within a small iteration budget and without any offline profiling. CORAL leverages distance covariance [16] to identify which hardware parameters most strongly influence throughput and power, and concentrates its limited search on those high-impact parameters.

1.3 Contributions

This thesis makes four contributions. The first is an exhaustive profiling study of the hardware-configuration space for Sat-NeRF training on the NVIDIA Jetson platform, spanning four satellite scenes, that establishes per-scene ORACLE configurations and quantifies the throughput–power–thermal trade-offs. Building on this study, the second contribution is a set of structural findings on the configuration landscape, notably that GPU frequency dominates throughput and power, that CPU core count has negligible impact, and that energy efficiency peaks at an intermediate rather than the maximum GPU frequency, together with a profiling-informed reduction of the search space. The third contribution is the application of CORAL [11], an online optimizer that selects hardware configurations during edge deep-learning training by using distance covariance to focus its limited search on the most influential hardware parameters, requiring no offline profiling. Finally, the fourth contribution is an empirical validation showing that CORAL discovers near-optimal configurations within roughly ten iterations, exploring on the order of one percent of the

configuration space and using an order of magnitude less device time than exhaustive profiling, and that it generalizes across scenes.

1.4 Structuring of Thesis

The remainder of this thesis is organized as follows. Chapter 2 provides the background on NeRF-based satellite 3D reconstruction, the edge hardware and its DVFS control surface, and the hardware-configuration optimization problem. Chapter 3 presents the methodology, formalizing the optimization problem and detailing the CORAL method and its algorithms. Chapter 4 reports the evaluation, including the exhaustive profiling landscape, the ORACLE configurations, the CORAL results against exhaustive-search baselines, and the search-space reduction. Chapter 5 discusses the findings and their limitations, and reviews the related work. Finally, Chapter 6 concludes the thesis and outlines directions for future work.

CHAPTER 2

BACKGROUND

This thesis sits at the intersection of two areas: neural 3D reconstruction from satellite imagery, which defines the *workload*, and the energy-efficient execution of deep learning on edge hardware, which defines the *optimization problem*. This chapter provides the background needed for both. It first describes satellite 3D reconstruction and the Sat-NeRF workload (Sections 2.1 and 2.2), then the edge hardware platform and its dynamic voltage and frequency scaling (DVFS) control surface (Sections 2.3 and 2.4), and finally frames the hardware-configuration optimization problem that Chapter 3 addresses (Section 2.5). Prior optimization methods are reviewed separately in Chapter 5.

2.1 Satellite 3D Reconstruction

Three-dimensional reconstruction of the Earth’s surface recovers the spatial structure of a scene from a set of overlapping two-dimensional images captured from different viewpoints, producing digital surface models and novel views of terrain, vegetation, and man-made structures. The classical pipeline combines Structure-from-Motion to estimate scene geometry and camera poses, multi-view stereo to densify it, and bundle adjustment to refine the result. While effective in many settings, these geometry-based methods struggle with texture-less regions, occlusions, ambiguous image features, and transient objects, and they often yield point clouds that are both sparse and inaccurate [13].

Satellite imagery adds a further difficulty in the form of camera modeling. Rather than a physical pinhole model, satellite sensors are described by the Rational Polynomial Coefficient (RPC) model, which relates 3D ground coordinates to image pixels through ratios of polynomials. Substituting a different camera model when reconstructing from satellite imagery introduces geometric approximations, so accurate pipelines instead refine the RPC parameters directly through bundle adjustment [8]. These limitations, sparse geometry-based output and sensitivity to the camera model, motivate a learned, volumetric representation that produces dense reconstructions and can incorporate the RPC model without approximation.

2.2 Neural Radiance Fields as a Training Workload

A Neural Radiance Field (NeRF) [9] represents a scene as a continuous volumetric function, realized by a multilayer perceptron, that maps a 3D point x and a viewing direction d to a color c and a volume density σ :

$$F : (x, d) \rightarrow (c, \sigma). \quad (2.1)$$

To render a pixel, points are sampled along the camera ray that passes through it, the network is evaluated at each sample, and the samples are composited front to back by volume rendering, weighting each color by its density and by the accumulated transmittance along the ray. Training minimizes the photometric error between rendered and observed pixels, iterating over batches of rays until the field converges. NeRF produces dense, photo-realistic reconstructions, but it learns a separate representation for each scene and therefore must be retrained from scratch for every new scene.

Satellite-specific variants adapt this formulation to overhead imagery (Figure 2.1). Shadow-NeRF [2] conditions the model on the solar direction and adds explicit shadow and sky modeling so that reconstructions remain consistent across images captured under different illumination. Sat-NeRF [7] unifies these ideas, ingests bundle-adjusted RPC camera parameters so that no approximating camera model is introduced, and models transient objects (such as vehicles and vegetation) through a per-image uncertainty term, with an optional solar-correction term and

depth supervision to stabilize and accelerate training. Sat-NeRF is the workload optimized in this thesis.

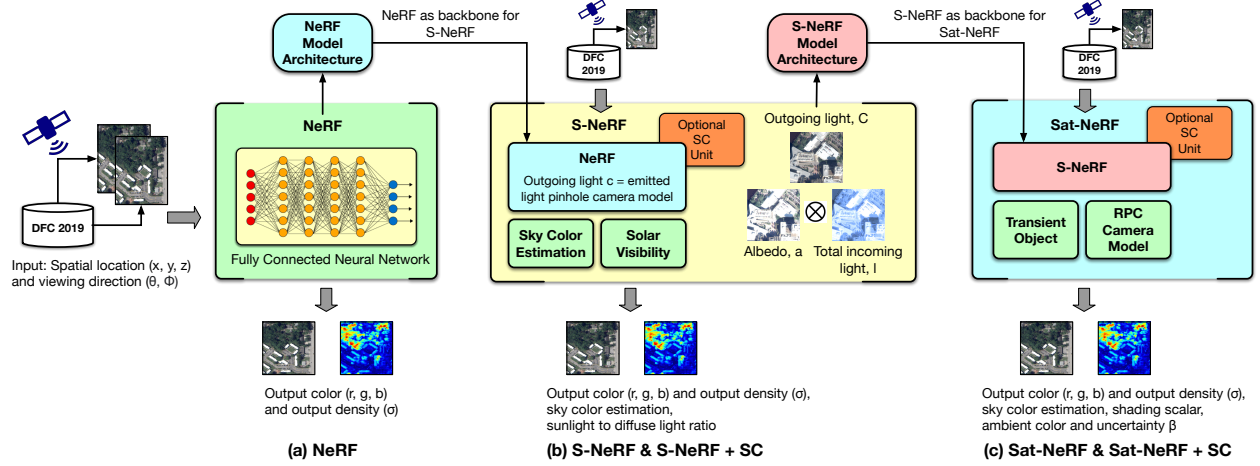


Figure 2.1: Overview of NeRF and its satellite variants. NeRF (a) uses a multilayer perceptron as its backbone. Shadow-NeRF (b) extends it with sky-color estimation, solar visibility, and an optional solar-correction unit for satellite imagery. Sat-NeRF (c) unifies the two and further handles bundle-adjusted RPC camera parameters and transient objects.

What matters for this thesis is Sat-NeRF viewed as a *training workload* rather than as a reconstruction model. Training is an iterative process of hundreds of thousands of ray-batch steps, each dominated by dense network evaluations and volume-rendering compositing on the GPU. Because the representation does not generalize across scenes, this cost is paid anew for every scene, and unlike a short inference pass it cannot be amortized over many repetitions. The result is a long, GPU-intensive computation that consumes substantial time and energy. On a data-center GPU this cost is a matter of convenience; on an edge device with a budget of a few watts it becomes the central constraint, and the hardware operating point at which training runs becomes a first-class concern.

2.3 Edge Computing and the NVIDIA Jetson Platform

Deploying deep learning at the *edge*, on devices close to where data is produced, reduces the latency and bandwidth of moving data to the cloud, keeps sensitive data local, and enables autonomous operation where connectivity is intermittent or unavailable. For satellite 3D recon-

struction, edge execution makes in-situ processing possible on platforms such as drones, ground stations, and small satellites, rather than requiring imagery to be downlinked and processed centrally.

The NVIDIA Jetson family brings GPU acceleration into this setting [12]. A Jetson module is a compact system-on-chip that integrates a multi-core CPU, a CUDA-capable GPU, and shared low-power memory within a small power envelope. The device used in this thesis is the Jetson Orin Nano 8GB. A defining property of such integrated designs is that the CPU and GPU share a single memory system and a single power and thermal budget, so they draw from and contend for the same envelope rather than being provisioned independently. This coupling is precisely what makes the choice of operating point consequential: raising the clock of one unit consumes budget that constrains the whole device. Moreover, because the platform dissipates heat through a fixed cooling solution, a sustained workload such as training drives the device toward its thermal limit in a way that brief inference bursts do not, making thermal behavior part of the problem rather than an afterthought.

2.4 Dynamic Voltage and Frequency Scaling and the Configuration Space

Edge platforms expose their power and performance trade-off through Dynamic Voltage and Frequency Scaling (DVFS), which adjusts the clock frequency, and the voltage that supports it, of the compute units at runtime. Because dynamic power grows faster than linearly with frequency, DVFS provides a direct lever for trading performance against power and heat. On the Jetson platform the relevant tunable hardware parameters are the number of active CPU cores (which can be brought online or offline), the CPU frequency, the GPU frequency, and the memory frequency. A *hardware configuration* is an assignment of values to these hardware parameters, and it fully determines the operating point at which a workload runs.

Configurations are controlled and observed through the platform’s standard tooling. The `nvpmodel` utility applies a configuration by bringing the requested cores online and setting the CPU and GPU frequency caps, while `jetson_clocks` pins the clocks to their maximum.

During execution, the `tegrastats` interface reports board power, GPU utilization, memory use, and component temperatures at roughly one-second intervals. From these, three quantities summarize an operating point: training *throughput*, measured as steps completed per unit wall-clock time (iterations per second); *power*, the mean board power draw over the run (watts); and *energy*, the integral of power over the training duration.

Each configuration therefore yields a throughput-power operating point, and the space of these points has a characteristic structure. Raising a frequency increases throughput but also increases power and temperature, and because these do not grow in proportion, the most energy-efficient operating point, the one that delivers the most training progress per watt, need not be the fastest one. Two different configurations can even deliver the same throughput at noticeably different power. Manufacturer power presets expose only a handful of coarse points within this space and cannot express the interior operating points where efficiency is often highest. Characterizing this space and selecting good points within it is the subject of this thesis.

2.5 Hardware-Configuration Optimization

Selecting a hardware configuration can be framed as an optimization problem: among all configurations in the space, find one that satisfies a performance requirement (for example, a minimum training throughput) while minimizing power or energy. Four properties make this problem difficult. First, the space is large, growing combinatorially with the number of hardware parameters and their levels. Second, the mapping from configuration to performance is non-linear, and the hardware parameters interact: raising the GPU frequency helps only when the CPU can supply data fast enough to keep the GPU busy, so per-parameter effects are not additive and the efficient region cannot be read off individual settings. Third, evaluation is black-box and costly: the only way to learn a configuration’s throughput and power is to run the workload and measure them, and for training each such measurement is an expensive partial run. Fourth, a feasibility constraint separates configurations that meet the performance requirement from those that do not.

Two broad paradigms address this problem. *Offline profiling* measures many or all configurations ahead of time and then selects the best; it is accurate but must be repeated for every

new model, device, or scene, consuming a large measurement budget. *Online adaptation* instead adjusts the configuration during operation, guided by a small number of live measurements; it is far cheaper but must converge within a tight budget. For a workload that is trained once per scene, spending a large profiling budget to tune a single run is wasteful, which motivates an online approach.

A key lever for making online search efficient is to reduce the effective dimensionality of the problem. If only a few hardware parameters materially influence performance, the search can concentrate its limited budget on those and hold the rest at power-saving settings. Identifying which hardware parameters matter requires a measure of the statistical dependence between each hardware parameter and the performance metrics. Linear correlation captures only monotone linear association and can therefore miss the non-linear, interacting dependencies that are typical of DVFS, so a measure that detects arbitrary non-linear dependence is preferable. Chapter 3 adopts distance covariance for exactly this purpose and details the online optimizer built upon it.

CHAPTER 3

METHODOLOGY

This chapter presents the online hardware-configuration optimizer used in this thesis [11], originally introduced for efficient edge inference. The optimizer discovers near-optimal operating points for a deep-learning workload on an edge device without exhaustive offline profiling. It leverages distance covariance to capture the non-linear dependence between hardware settings and performance, and uses those dependence weights to guide a lightweight search that converges within a small iteration budget. We first formalize the optimization problem (Section 3.1), review the distance-covariance dependence measure (Section 3.2), and then present the method as a flowchart and describe its component steps (Sections 3.3–3.6).

Two aspects of the method are highlighted where they occur: the search direction is keyed on the *incumbent* configuration’s throughput (Section 3.6), and the pinning of non-influential hardware parameters is derived from the measured correlation structure rather than hardcoded to a specific hardware parameter.

3.1 Problem Formulation

A hardware configuration is a vector of tunable settings. On the Jetson platform used in this thesis, a configuration is the triple

$$s = (c_{\text{cpu}}, f_{\text{cpu}}, f_{\text{gpu}}), \quad (3.1)$$

where c_{cpu} is the number of active CPU cores, f_{cpu} is the CPU frequency, and f_{gpu} is the GPU frequency. Each configuration s yields a measured training throughput $\tau(s)$ (iterations per second) and average board power $p(s)$ (watts). The discretized space S contains all combinations of the per-parameter levels.

Given a throughput target τ_{target} , we seek a configuration s^* that meets the target while maximizing energy efficiency:

$$s^* = \arg \max_{s \in S} \eta(s) \quad \text{subject to} \quad \tau(s) \geq \tau_{\text{target}}, \quad \eta(s) = \frac{\tau(s)}{p(s)}. \quad (3.2)$$

A configuration that fails to meet the target is *infeasible* and is excluded from further exploration. Among feasible configurations, higher efficiency $\eta(s)$ means more training progress per watt, which is desirable for power- and thermally-constrained edge deployment.

This problem is difficult for three reasons. First, the configuration space is large: even the three-parameter space in this thesis contains hundreds of configurations, and general edge platforms expose thousands. Second, $\tau(s)$ and $p(s)$ are non-linear and interact: raising GPU frequency improves throughput only when the CPU is fast enough to feed it. Third, the feasible region is often narrow, so exhaustive search is impractical. These challenges motivate a dependence-guided search that identifies the high-impact hardware parameters without profiling the entire space.

3.2 Distance Covariance

The optimizer relies on *distance covariance* [16], a statistical measure of dependence between two variables that, unlike Pearson correlation, detects arbitrary non-linear relationships and is zero if and only if the variables are independent. This makes it well suited to the non-linear hardware–performance relationship.

Given n paired observations of a performance metric m (throughput or power) and a hardware setting s , we form the pairwise Euclidean distance matrices

$$a_{ij} = \|m_i - m_j\|, \quad b_{ij} = \|s_i - s_j\|, \quad (3.3)$$

and double-center them to remove marginal effects:

$$A_{ij} = a_{ij} - \bar{a}_{i\cdot} - \bar{a}_{\cdot j} + \bar{a}_{\cdot\cdot}, \quad B_{ij} = b_{ij} - \bar{b}_{i\cdot} - \bar{b}_{\cdot j} + \bar{b}_{\cdot\cdot}, \quad (3.4)$$

where $\bar{a}_{i\cdot}$, $\bar{a}_{\cdot j}$, and $\bar{a}_{\cdot\cdot}$ are the row, column, and grand means. The (squared) distance covariance and its normalized form, the distance correlation, are

$$\text{dCov}^2(m, s) = \frac{1}{n^2} \sum_{i,j} A_{ij} B_{ij}, \quad \text{dCor}(m, s) = \frac{\text{dCov}(m, s)}{\sqrt{\text{dCov}(m, m) \text{dCov}(s, s)}}. \quad (3.5)$$

The distance correlation $\text{dCor} \in [0, 1]$ equals 0 for independent variables and approaches 1 for strong dependence. The optimizer uses the normalized form so that dependence weights are comparable across hardware parameters of different units and ranges.

3.3 Overview of the Optimizer

Figures 3.1 and 3.2 present the optimizer’s procedure as a flowchart, split into its exploration and exploitation stages; the numbered steps are referenced throughout this chapter. The optimizer searches the configuration space in three phases. In the **exploration** phase (steps 1–4, Figure 3.1), the optimizer starts from a maximum-performance reference configuration (step 1) and evaluates a small set of diverse configurations chosen by Latin Hypercube Sampling (LHS, step 4). Each evaluation runs one fixed training session, measures throughput and power (step 2), and assigns an efficiency score (step 3); together these seed the dependence estimate with observations spread across the space. In the **exploitation** phase (steps 5–9, Figure 3.2), the optimizer anchors on the best-scoring configurations (step 5), estimates how strongly each hardware parameter influences throughput and power (step 6), and proposes the next configuration by adjusting high-influence parameters more aggressively than low-influence ones (steps 7–8). It re-scores the proposal (step 9) and repeats, sharpening its influence estimates while the search improves and perturbing the search direction when it stalls. Finally, in the **testing** phase (step 10, Figure 3.2) the best configuration is re-evaluated to confirm its performance under measurement noise, and the optimizer returns it.

Two conditions govern how long the search runs. The exploitation loop continues until a fixed iteration budget is exhausted or no promising candidate remains; the full budget in this thesis is ten iterations, which is sufficient for convergence while remaining practical for online adaptation. Across the run the optimizer also adapts the throughput target gently: after a feasible streak it nudges τ_{target} upward to seek higher-throughput operating points, and relaxes it if the target proves unreachable, keeping the search anchored in the feasible region. The three building blocks invoked by these phases, the efficiency score (step 3, Figure 3.1), the correlation analysis (step 6), and the configuration search (steps 7–8, both in Figure 3.2), are detailed in the following sections.

3.4 Efficiency and Feasibility

Every evaluated configuration receives a single efficiency score that couples the feasibility constraint to the throughput objective (step 3 in Figure 3.1). A *feasible* configuration, one that meets the throughput target, is scored by its efficiency τ/p ; an *infeasible* one receives the negated inverse ratio $-p/\tau$ and is added to a prohibited set PS :

$$r(s) = \begin{cases} \tau(s)/p(s) & \text{if } \tau(s) \geq \tau_{\text{target}} \text{ (feasible),} \\ -p(s)/\tau(s) & \text{otherwise (infeasible).} \end{cases} \quad (3.6)$$

The penalty guarantees that every infeasible configuration ranks below every feasible one, and its inverted form further penalizes settings that draw power without delivering throughput. Before evaluating a candidate, the optimizer checks PS and skips any previously prohibited configuration to avoid redundant runs.

3.5 Correlation Analysis

Using the most recent W observations (the sliding window), the optimizer computes, for each hardware parameter i , two dependence weights (step 6 in Figure 3.2): the distance correlation

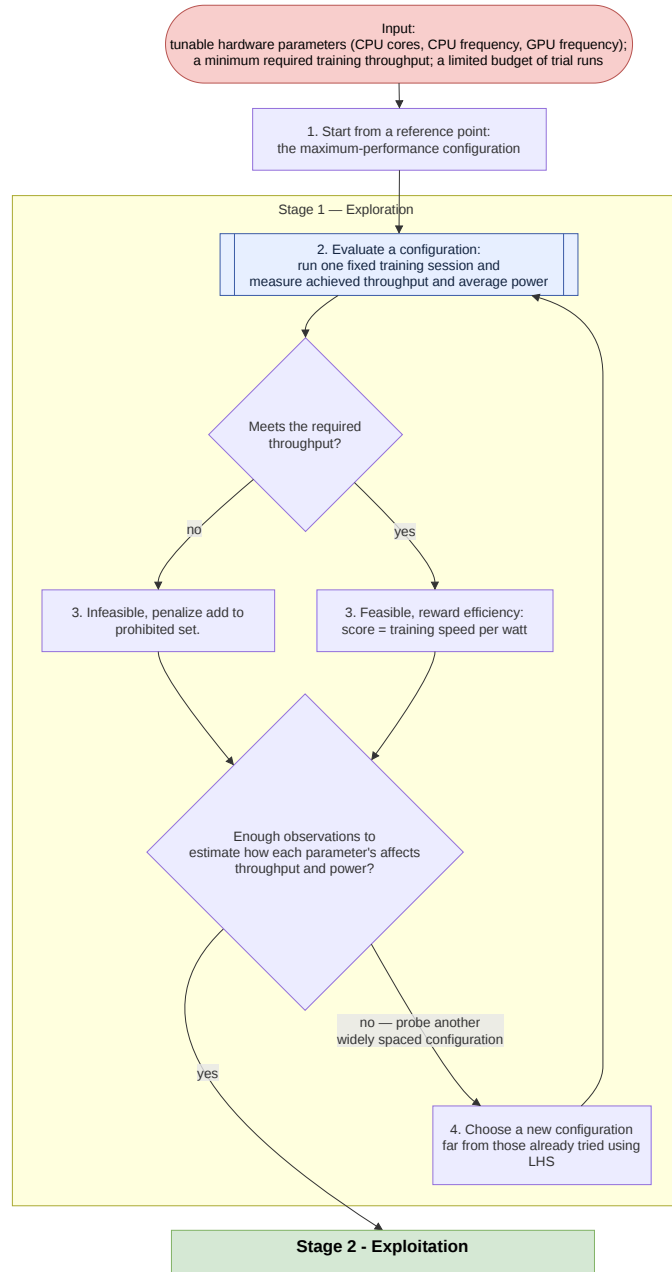


Figure 3.1: The optimizer, exploration phase (steps 1–4). Starting from a maximum-performance reference configuration (step 1), the optimizer samples diverse configurations by Latin Hypercube Sampling (step 4) and scores each one (steps 2–3): a feasible configuration (meeting the throughput target) is scored by its efficiency, while an infeasible one is penalized and avoided. Exploration continues until enough configurations have been seen to estimate how each hardware parameter affects throughput and power, after which the search proceeds to the exploitation phase (Figure 3.2).

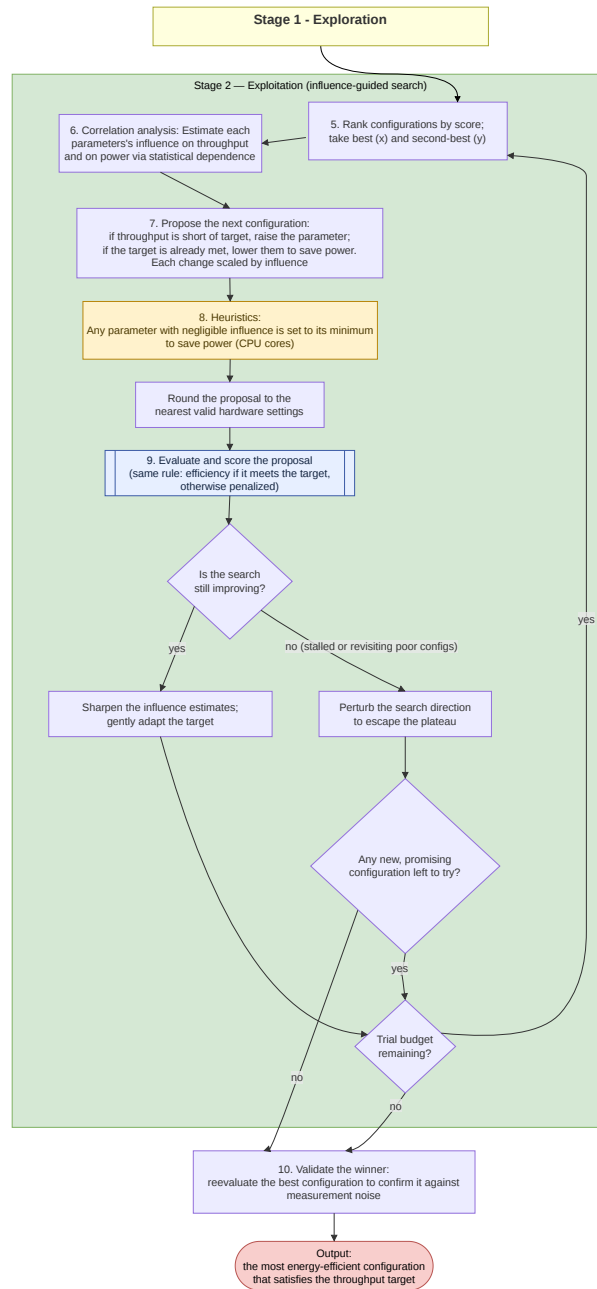


Figure 3.2: The optimizer, exploitation phase (steps 5–10), continuing from exploration (Figure 3.1). The optimizer anchors on the best-scoring configurations (step 5), estimates each hardware parameter’s influence on throughput and power (step 6), and proposes the next configuration (steps 7–8) by adjusting high-influence parameters more aggressively than low-influence ones, scaling each step by that parameter’s influence and pinning parameters with negligible influence (here, CPU cores) to their minimum to save power (step 8). The loop refines the influence estimates while the search improves and perturbs it when it stalls (step 9), and validates the winning configuration once the budget is spent (step 10).

with throughput and with power,

$$\alpha_i = \text{dCor}(\tau, s_i), \quad \beta_i = \text{dCor}(p, s_i). \quad (3.7)$$

A high α_i means hardware parameter i strongly drives throughput; a high β_i means it strongly drives power. The optimizer takes the dominant weight $\gamma_i = \max(\alpha_i, \beta_i)$ to scale how aggressively that hardware parameter is adjusted: influential hardware parameters receive large steps, while hardware parameters with $\gamma_i \approx 0$ are treated as inert. The sliding window keeps the computation lightweight and lets the weights adapt as the search progresses.

3.6 Configuration Search

The configuration search generates the next configuration from the best (x) and second-best (y) configurations observed so far (steps 7–8 in Figure 3.2). For each hardware parameter i it takes a step

$$\Delta_i = \frac{1}{2} |x_i - y_i| \cdot \gamma_i, \quad (3.8)$$

proportional to both the spread between the two best configurations and the parameter’s correlation weight γ_i , so influential parameters move further. The step *direction* is set by the incumbent configuration: if the best configuration already meets the throughput target, the optimizer moves toward lower settings to reduce power; otherwise it moves toward higher settings to raise throughput,

$$v_i = \begin{cases} l_i - \Delta_i & \text{if } \tau_x \geq \tau_{\text{target}} \text{ (incumbent feasible: reduce power),} \\ h_i + \Delta_i & \text{otherwise (incumbent infeasible: raise throughput),} \end{cases} \quad (3.9)$$

where $(l_i, h_i) = (x_i, y_i)$ normally and (y_i, x_i) when the search is directed to the opposite side of the incumbent. Keying the direction on the incumbent’s throughput τ_x is the correct criterion: it reflects whether the configuration the optimizer is actually refining is feasible. Each value v_i is rounded and clamped to the parameter’s valid range. When a move is blocked (the candidate is

prohibited or already visited), the optimizer searches the opposite side of the incumbent, and falls back to a random perturbation if both sides are exhausted.

Finally, once the target is met, any hardware parameter whose correlation is negligible ($\gamma_i \approx 0$) contributes no throughput and is pinned to its minimum to save power (step 8 in Figure 3.2). In this thesis the CPU core count is consistently identified as such a parameter (its distance correlation with throughput is ≈ 0), so it is driven to its minimum; but this decision is *derived from the measured correlation structure* rather than hardcoded to a particular parameter, so the same rule applies unchanged on workloads where a different parameter is inert.

Because the optimizer prioritizes high-correlation hardware parameters and skips prohibited configurations, it concentrates its limited budget on the parameters that actually move throughput and power, converging to near-optimal operating points in an order of magnitude fewer evaluations than exhaustive profiling. Chapter 4 evaluates this claim against exhaustive-search ORACLE baselines across four satellite scenes.

CHAPTER 4

EVALUATION

This chapter evaluates hardware configuration optimization for training two neural radiance field workloads, Sat-NeRF and S-NeRF, on the NVIDIA Jetson Orin Nano 8GB edge device. For each workload we characterize the effect of hardware parameters on performance through exhaustive profiling of the configuration space across four satellite scenes, identifying the optimal (ORACLE) configuration for each scene and the structural relationships between hardware parameters and performance. We then validate the optimizer on both workloads, showing that it discovers near-optimal configurations at a small fraction of the cost of exhaustive search, and finally quantify the benefit of a profiling-informed search space reduction.

4.1 Experimental Setup

Following a common evaluation methodology, we describe the workloads (the trained DL models), the measurement setup, the configuration space, the dataset, and the evaluation procedure. Because our workloads are neural radiance field *training* rather than a short, repeatable inference pass, each configuration is evaluated by a reduced training run rather than a single forward pass.

4.1.1 DL Workloads

We evaluate two satellite neural radiance field training workloads. **Sat-NeRF** reconstructs scene radiance and altitude from multi-view satellite imagery, using a fully-connected backbone of 8 lay-

ers with 256 units, 64 coarse samples per ray, and auxiliary albedo, transparency, and shadow/solar components together with a transient-embedding and a per-ray uncertainty head. **S-NeRF** is the shadow-aware predecessor: it shares the same fully-connected backbone and shadow/solar rendering but omits the transient-embedding and uncertainty heads, making it a distinct but comparable workload on the same hardware. Both are implemented in PyTorch Lightning and trained with BFloat16 mixed precision. Each configuration is evaluated with a reduced training run of 2,580 steps at a batch size of 2,048 rays, which is sufficient to reach a stable throughput, power, and thermal operating point. Reconstruction quality is measured by PSNR and altitude MAE (Section 4.4); because these depend only on the scene and training budget and not on the hardware configuration, throughput (iterations per second) and power are the optimization objectives for both workloads.

4.1.2 Measurement Setup

All experiments run on a single NVIDIA Jetson Orin Nano 8GB. We collect system-level metrics with the standard Jetson tooling: `tegrastats` for board power (VDD_IN), GPU utilization, and memory, and the `sysstat/pidstat` utilities for CPU and memory utilization. Metrics are sampled at 1 Hz throughout each training run, synchronized with throughput measurement. Throughput, the primary performance metric, is defined as the number of training iterations (steps) completed per second (iter/s), computed as the step count divided by the wall-clock training duration; power is the mean board power over the run (W); and energy is obtained by rectangle-rule integration of the power trace. Hardware configurations are applied through the Jetson `nvpmodel` utility, which brings the requested CPU cores online and sets the CPU and GPU frequency caps before each run.

4.1.3 Configuration Space

Each configuration is a triple $s = (c_{\text{cpu}}, f_{\text{cpu}}, f_{\text{gpu}})$ of active CPU cores, CPU frequency, and GPU frequency, and yields a throughput $\tau(s)$ and power $p(s)$. Although the platform also exposes memory frequency and concurrency level, we hold memory frequency at the platform maximum

and omit concurrency because both training workloads are single-instance. We discretize the ranges with practical step sizes (~ 100 MHz for CPU and GPU frequency, and one core), giving $5 \times 15 \times 8 = 600$ configurations per scene, summarized in Table 4.1.

Table 4.1: Hardware configuration space. All 600 combinations were profiled exhaustively for each scene.

Hardware parameter	Range	Levels	Step
CPU cores	2 – 6	5	1 core
CPU frequency	316.8 – 1,728.0 MHz	15	100.8 MHz
GPU frequency	306 – 1,020 MHz	8	~ 102 MHz

To rank configurations we use their energy efficiency $\eta(s) = \tau(s)/p(s)$ subject to a throughput target τ_{target} : a configuration is *feasible* if it meets the target ($\tau(s) \geq \tau_{\text{target}}$), feasible configurations are ranked by their efficiency $\eta(s)$, and infeasible ones are ranked below all feasible ones (by $-p(s)/\tau(s)$). The ORACLE for a scene is the highest-efficiency feasible configuration in that scene’s exhaustive profiling. Because the two workloads have different throughput envelopes, we use $\tau_{\text{target}} = 3.0$ iter/s for Sat-NeRF and $\tau_{\text{target}} = 1.0$ iter/s for the lower-throughput S-NeRF; in both cases every ORACLE and every reported configuration is feasible, so the reported score is the energy efficiency τ/p .

4.1.4 Dataset

The input imagery is drawn from the DFC2019 multi-view satellite dataset. We evaluate four areas of interest (AOIs), all in Jacksonville: JAX_214, JAX_260, JAX_068, and JAX_004. All scenes share identical training parameters; only the input imagery differs. Each of the two workloads was profiled over all 600 configurations across the four scenes, giving 2,400 measurements per workload and 4,800 in total.

4.1.5 Evaluation Procedure

We implement the exhaustive profiling and the optimizer within a common evaluation loop. The loop initializes a search strategy (exhaustive search for the ORACLE, or the optimizer), which selects a hardware configuration; the configuration is applied via `nvpmodel`; training is executed

for 2,580 steps; and the resulting throughput and power are measured and fed back to the optimizer. For the ORACLE, the loop enumerates the full 600-configuration space per scene to establish the upper-bound baseline. For the optimizer, the loop repeats for a fixed budget of 10 iterations (episodes), which is sufficient for convergence while remaining practical for online adaptation. The optimizer is assessed by the efficiency, the rank of its selected configuration against the profiled distribution, the gap to the ORACLE, and the device time consumed (Section 4.5).

4.2 Hardware Parameter Characterization

4.2.1 Hardware Parameter Impact

Tables 4.2 and 4.3 report the Pearson correlation between each hardware parameter and the two primary metrics, per scene and workload. The hierarchy is the same for both workloads: GPU frequency dominates, with $r > 0.96$ for Sat-NeRF and $r \approx 0.82$ – 0.92 for S-NeRF on both throughput and power, while CPU core count is statistically indistinguishable from noise ($r \approx 0.00$ with throughput). CPU frequency has a weak positive influence that is somewhat larger for S-NeRF (up to ~ 0.34 on throughput and ~ 0.43 on power) than for Sat-NeRF, reflecting S-NeRF’s slightly higher sensitivity to host-side work, but it remains clearly secondary to GPU frequency. Figures 4.1–4.6 visualize this dominance for both workloads: throughput rises steeply with GPU frequency, only weakly with CPU frequency, and is flat across core counts.

Table 4.2: Pearson correlation between each hardware parameter and throughput / power, per scene (Sat-NeRF). GPU frequency dominates; CPU cores have near-zero impact on throughput.

Hardware parameter (metric)	JAX_214	JAX_260	JAX_068	JAX_004
GPU freq (iter/s)	0.972	0.968	0.972	0.963
CPU freq (iter/s)	0.149	0.161	0.151	0.161
CPU cores (iter/s)	-0.001	-0.001	-0.000	-0.017
GPU freq (power)	0.967	0.965	0.966	0.961
CPU freq (power)	0.223	0.229	0.225	0.221
CPU cores (power)	0.034	0.037	0.036	0.020

Table 4.3: Pearson correlation between each hardware parameter and throughput / power, per scene (S-NeRF). GPU frequency dominates; CPU cores have near-zero impact on throughput.

Hardware parameter (metric)	JAX_214	JAX_260	JAX_068	JAX_004
GPU freq (iter/s)	0.817	0.886	0.917	0.890
CPU freq (iter/s)	0.341	0.249	0.188	0.243
CPU cores (iter/s)	-0.000	0.001	-0.003	0.002
GPU freq (power)	0.828	0.865	0.897	0.874
CPU freq (power)	0.432	0.375	0.339	0.361
CPU cores (power)	0.032	0.024	0.032	0.035

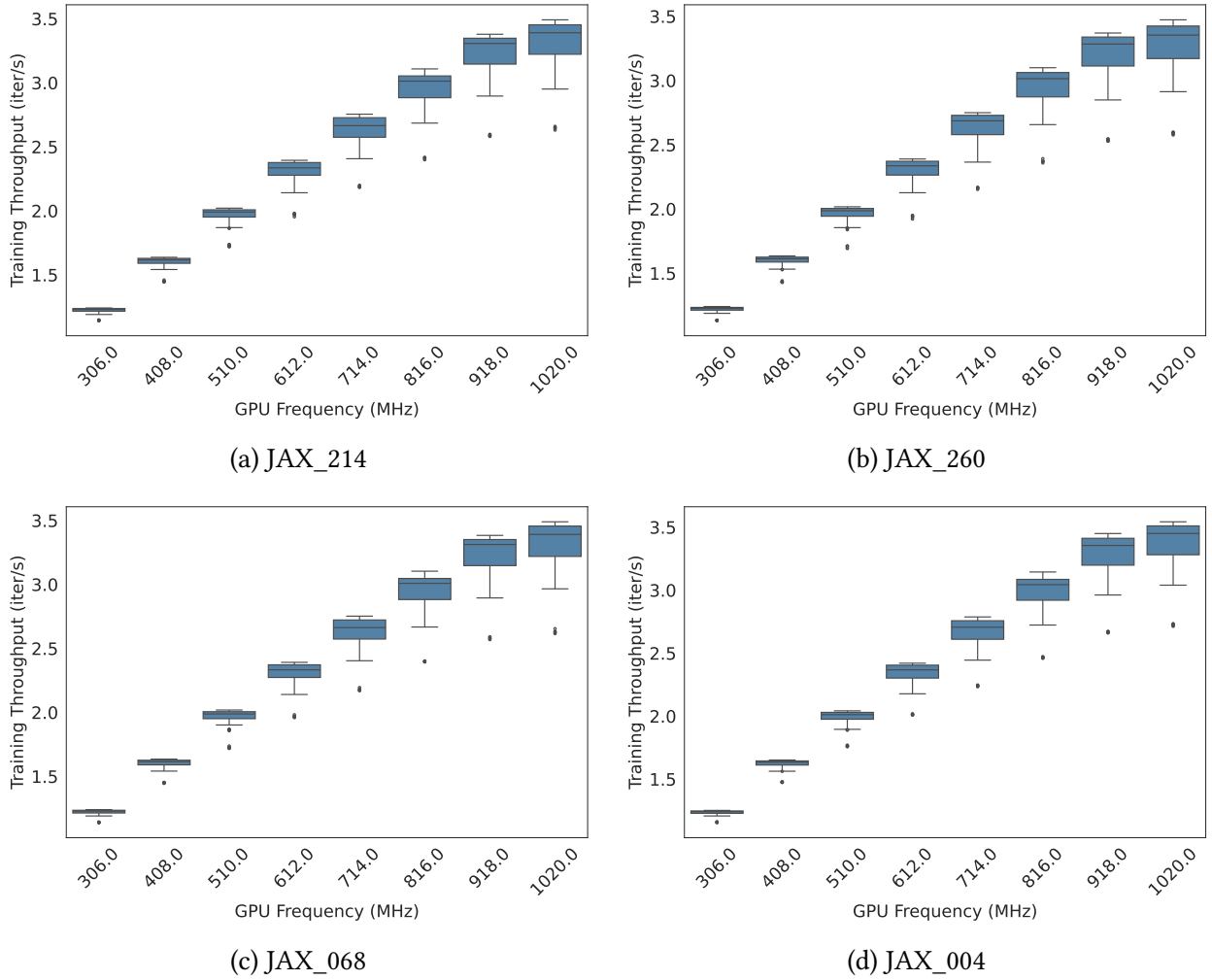
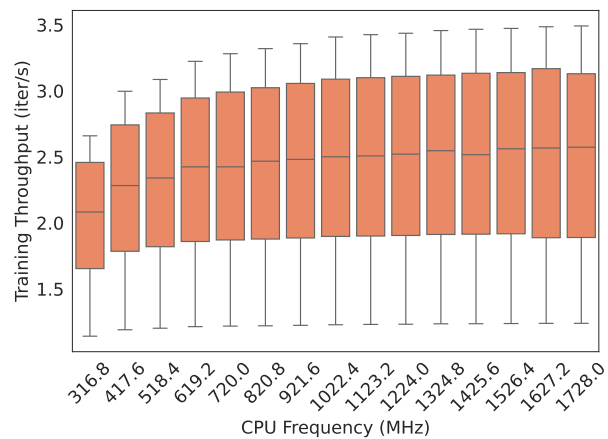
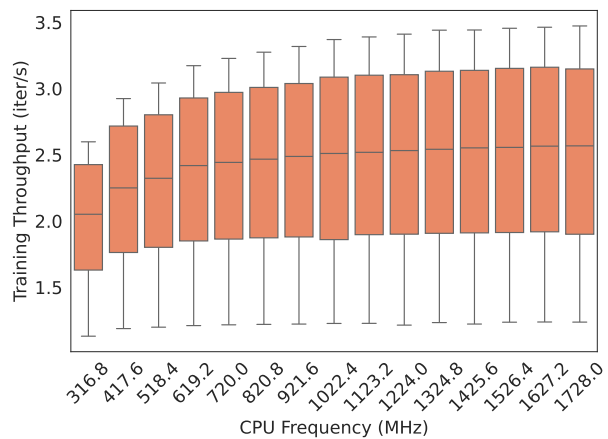


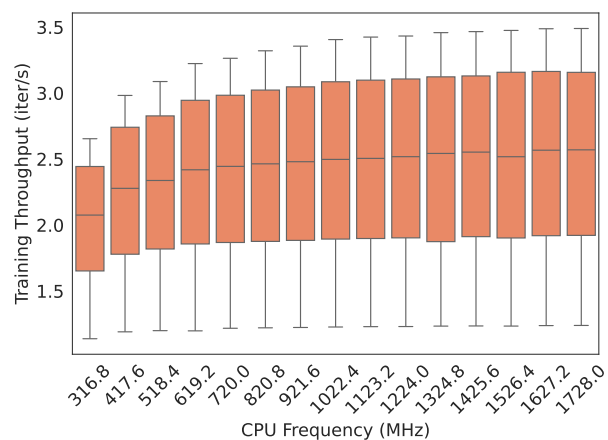
Figure 4.1: Sat-NeRF: training throughput versus GPU frequency.



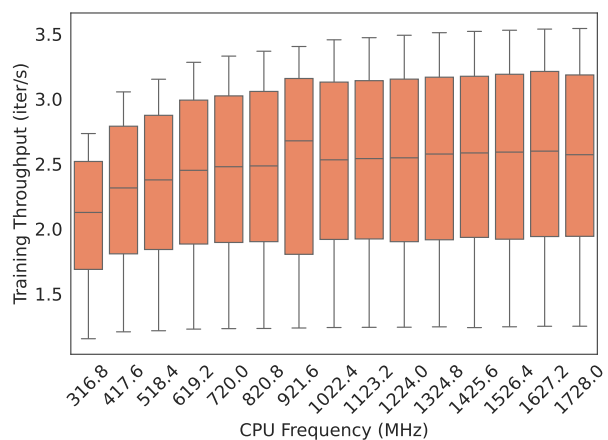
(a) JAX_214



(b) JAX_260

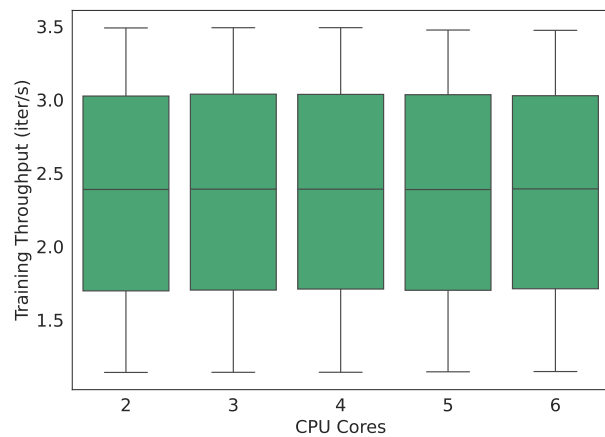


(c) JAX_068

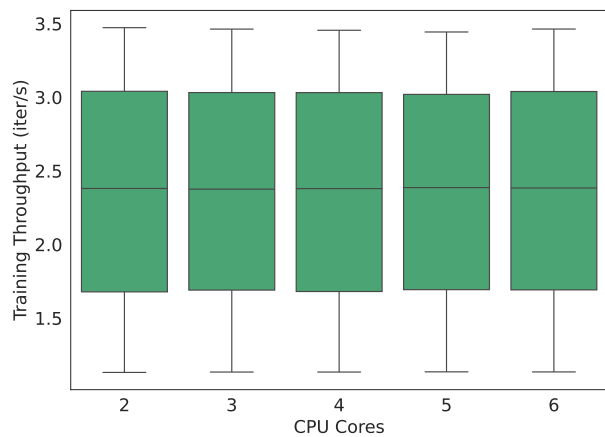


(d) JAX_004

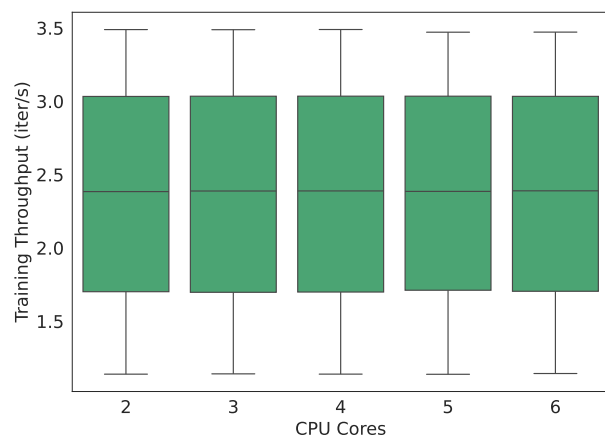
Figure 4.2: Sat-NeRF: training throughput versus CPU frequency.



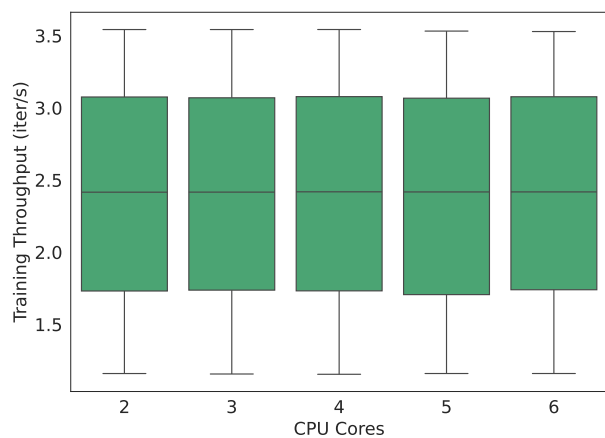
(a) JAX_214



(b) JAX_260

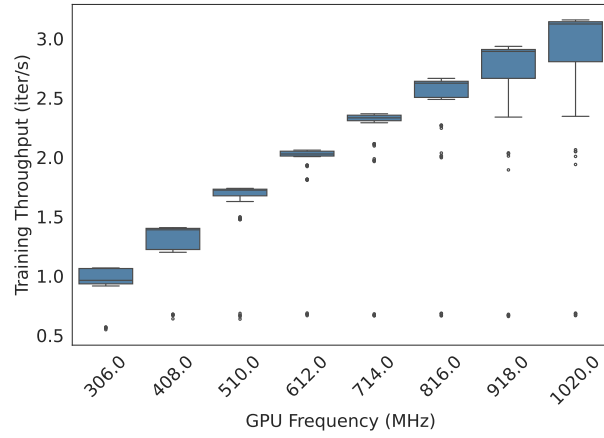


(c) JAX_068

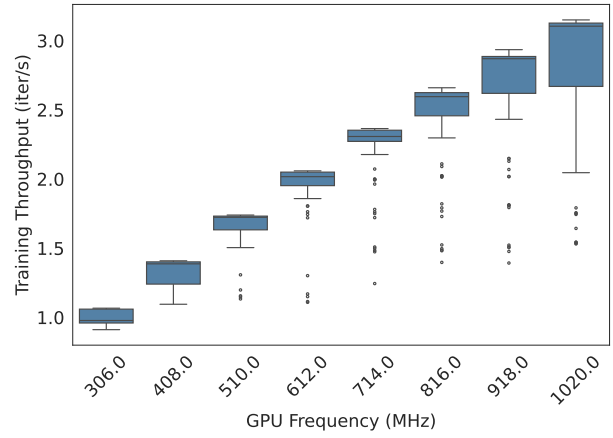


(d) JAX_004

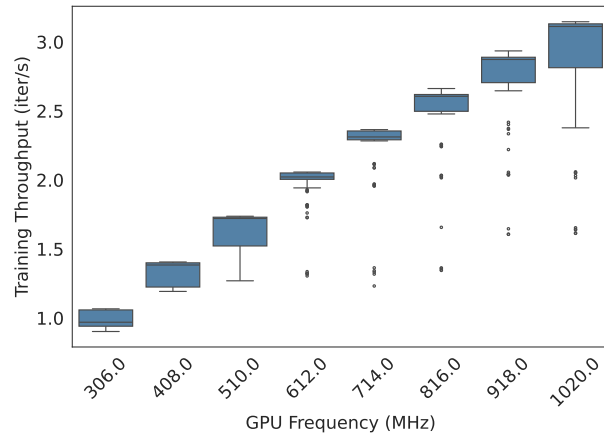
Figure 4.3: Sat-NeRF: training throughput versus CPU core count.



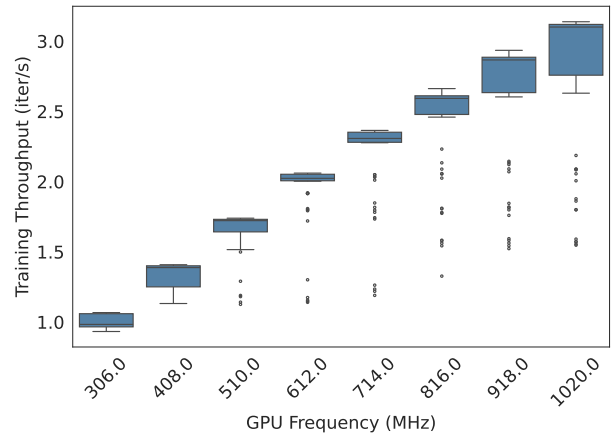
(a) JAX_214



(b) JAX_260

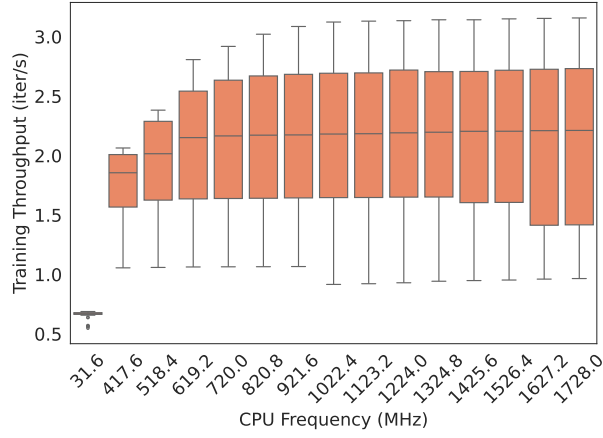


(c) JAX_068

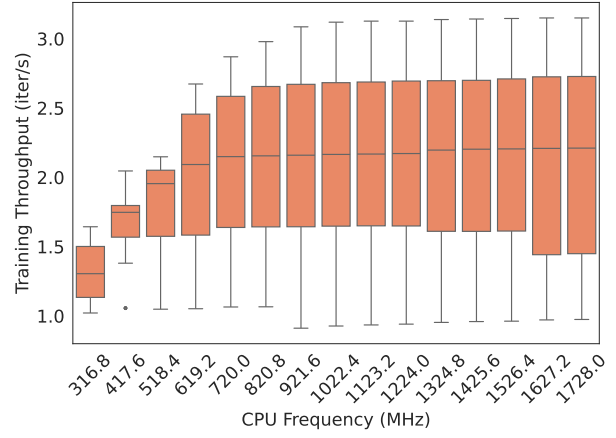


(d) JAX_004

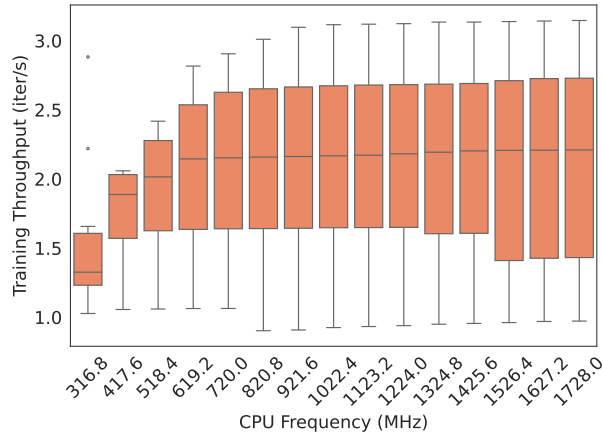
Figure 4.4: S-NeRF: training throughput versus GPU frequency.



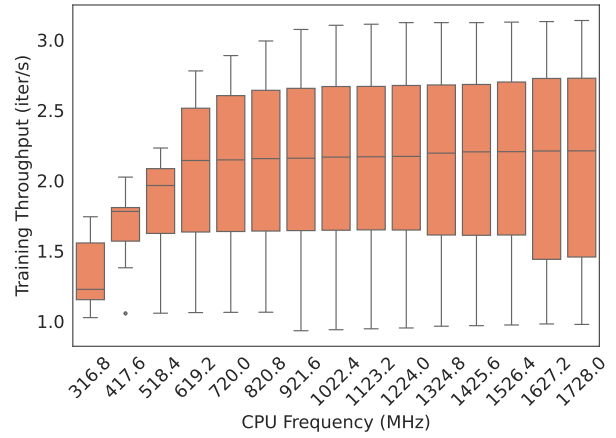
(a) JAX_214



(b) JAX_260



(c) JAX_068



(d) JAX_004

Figure 4.5: S-NeRF: training throughput versus CPU frequency.

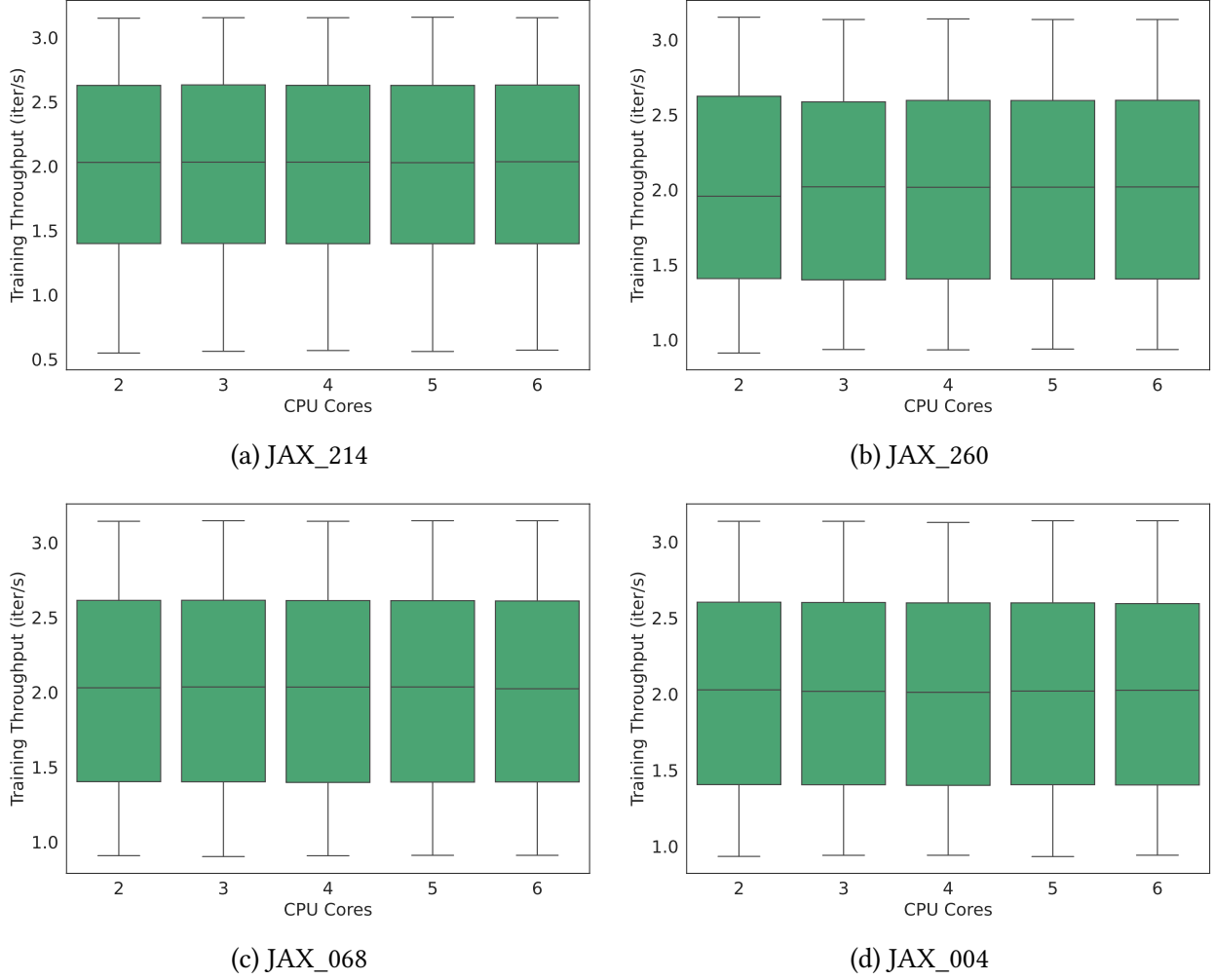


Figure 4.6: S-NeRF: training throughput versus CPU core count.

4.2.2 Throughput–Power Trade-off and Energy Efficiency

Because GPU frequency governs both throughput and power, each workload’s configuration space traces a clear throughput–power front (Sat-NeRF: Figure 4.7; S-NeRF: Figure 4.11), colored by GPU frequency. Sat-NeRF throughput varies roughly $3\times$ (from ~ 1.1 to ~ 3.5 iter/s) at $\sim 9.5\text{--}20$ W, and S-NeRF spans a similar $\sim 0.9\text{--}3.2$ iter/s at a lower $\sim 7.5\text{--}17$ W (Tables 4.4 and 4.5). For both workloads the maximum GPU frequency is *not* the most energy-efficient choice: energy efficiency (iter/s per watt) peaks at an intermediate GPU frequency and declines at 1,020 MHz (Sat-NeRF: Figure 4.8; S-NeRF: Figure 4.12), and the projected full-training energy confirms that this efficiency-optimal frequency also minimizes total training energy (Figure 4.9 for Sat-NeRF

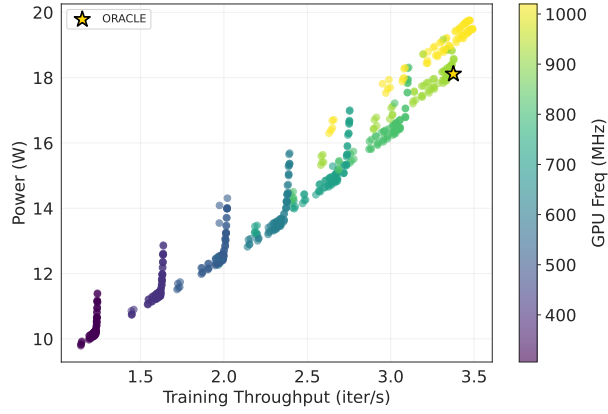
and 4.13 for S-NeRF). The efficiency peak sits at an intermediate 816–918 MHz for both workloads. The corresponding Pareto frontiers are shown in Figure 4.10 (Sat-NeRF) and 4.14 (S-NeRF).

Table 4.4: Per-scene ranges of throughput and power, and the best (ORACLE) efficiency (Sat-NeRF, target 3.0 iter/s). The hardware performance envelope is nearly scene-invariant.

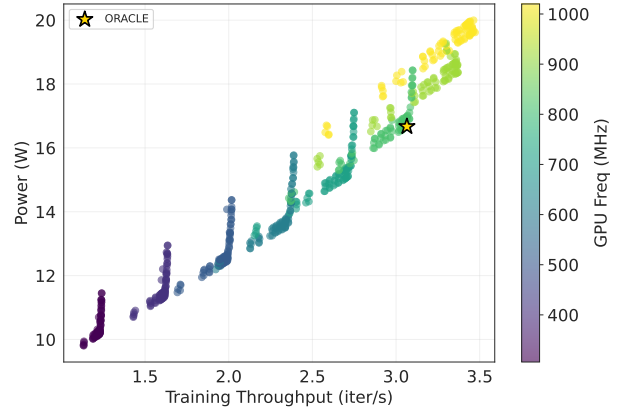
Scene	iter/s (min/mean/max)	Power W (min/mean/max)	Best efficiency
JAX_214	1.14/2.39/3.49	9.8/14.5/19.8	0.1864
JAX_260	1.13/2.38/3.47	9.8/14.7/20.0	0.1839
JAX_068	1.14/2.39/3.49	9.8/14.5/19.8	0.1868
JAX_004	1.10/2.42/3.54	9.6/14.4/19.8	0.1903

Table 4.5: Per-scene ranges of throughput and power, and the best (ORACLE) efficiency (S-NeRF, target 1.0 iter/s). S-NeRF tops out below Sat-NeRF, so its feasible target is 1.0 iter/s and its best efficiency is the peak τ/p over the space.

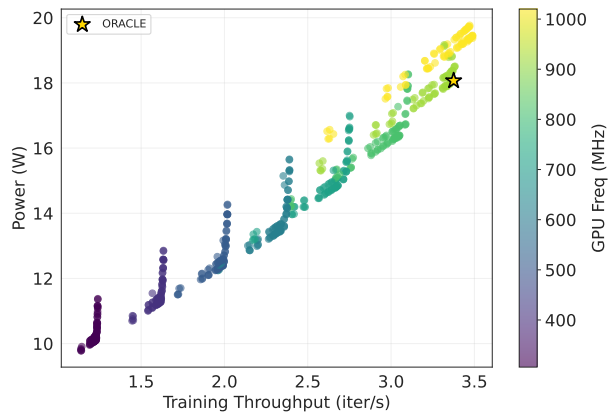
Scene	iter/s (min/mean/max)	Power W (min/mean/max)	Best efficiency
JAX_214	0.55/1.98/3.16	7.3/12.2/17.4	0.1916
JAX_260	0.91/2.00/3.15	7.6/12.0/17.0	0.1957
JAX_068	0.90/2.02/3.15	8.7/12.2/17.0	0.1934
JAX_004	0.93/2.01/3.14	8.6/12.0/16.9	0.1952



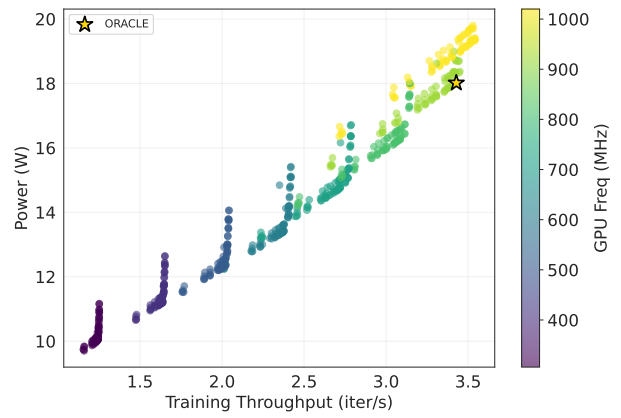
(a) JAX_214



(b) JAX_260

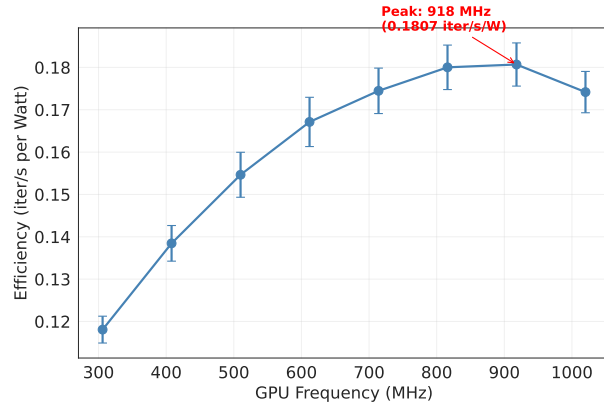


(c) JAX_068

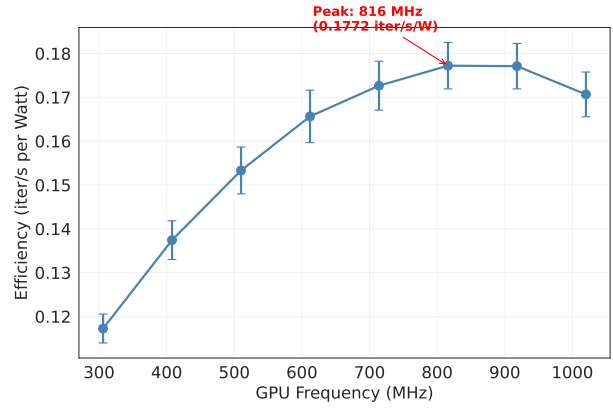


(d) JAX_004

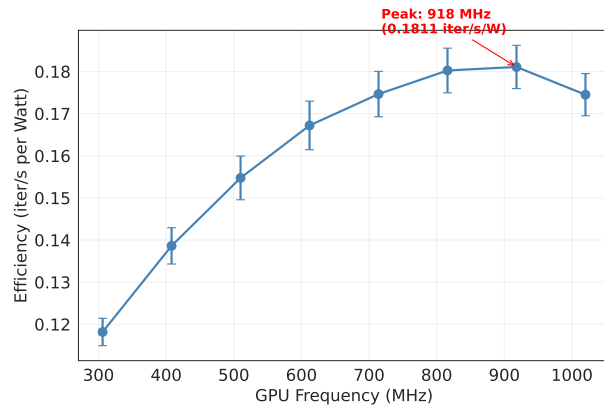
Figure 4.7: Sat-NeRF: throughput versus power for all profiled configurations, colored by GPU frequency (ORACLE starred).



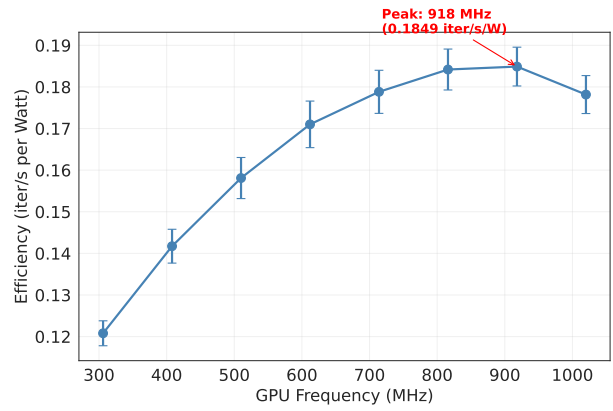
(a) JAX_214



(b) JAX_260

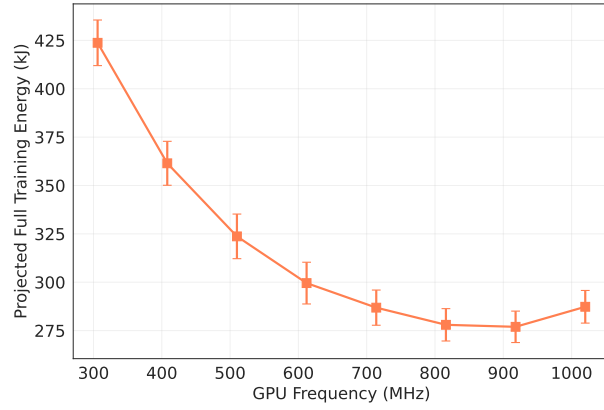


(c) JAX_068

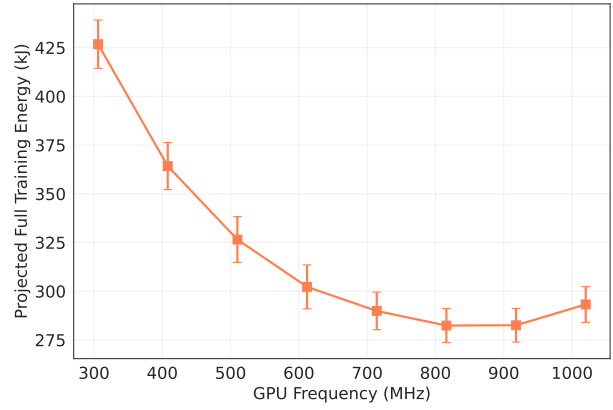


(d) JAX_004

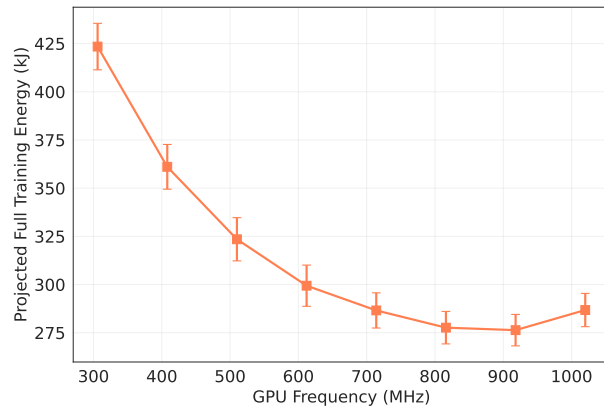
Figure 4.8: Sat-NeRF: energy efficiency (iter/s per watt) versus GPU frequency; efficiency peaks at an intermediate frequency.



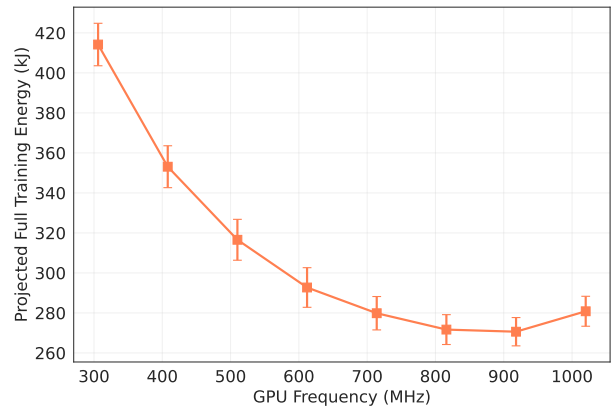
(a) JAX_214



(b) JAX_260

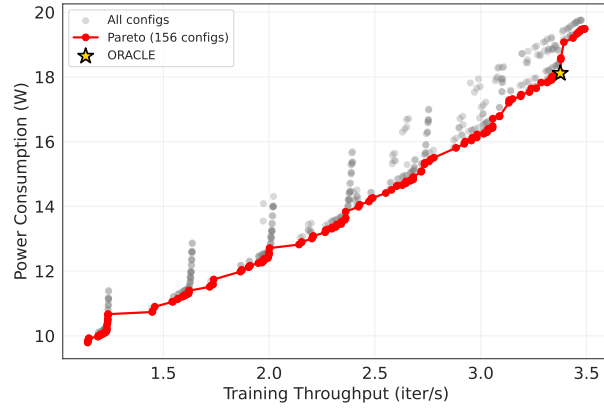


(c) JAX_068

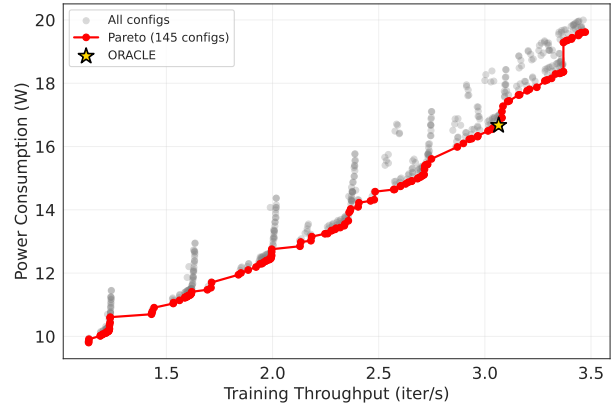


(d) JAX_004

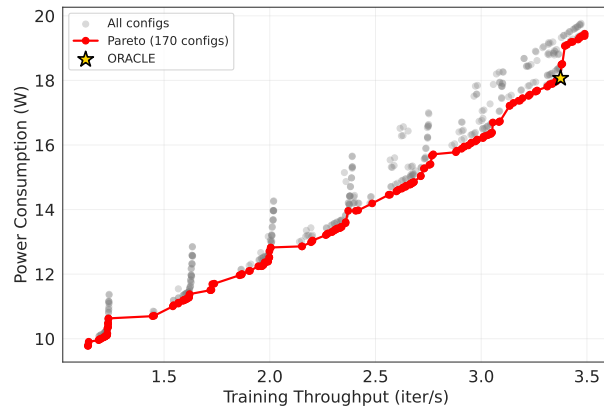
Figure 4.9: Sat-NeRF: projected full-training energy (50,000 steps) versus GPU frequency; energy is minimized at the intermediate efficiency-peak frequency.



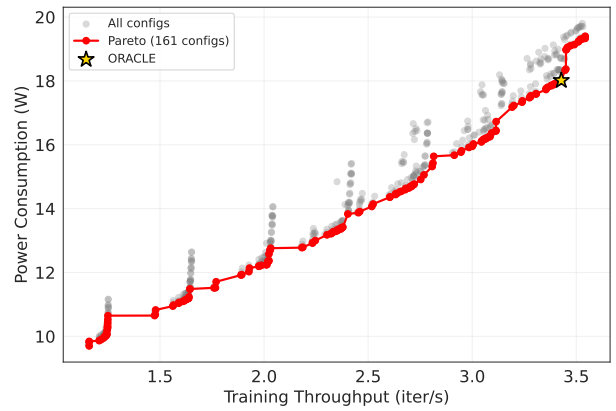
(a) JAX_214



(b) JAX_260

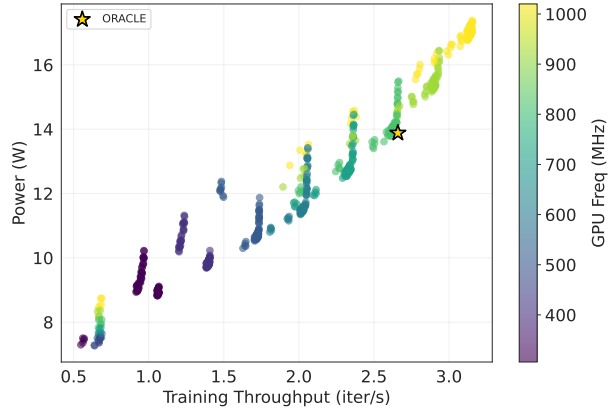


(c) JAX_068

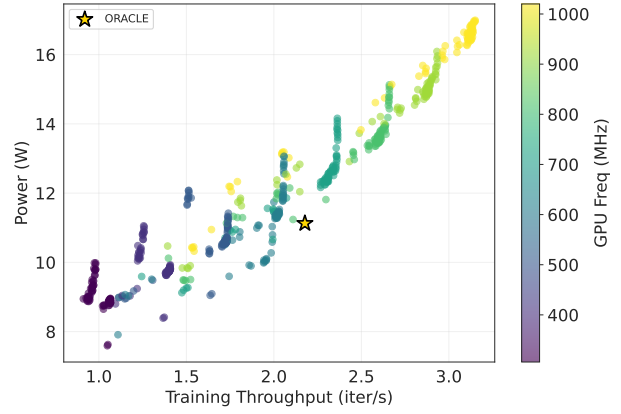


(d) JAX_004

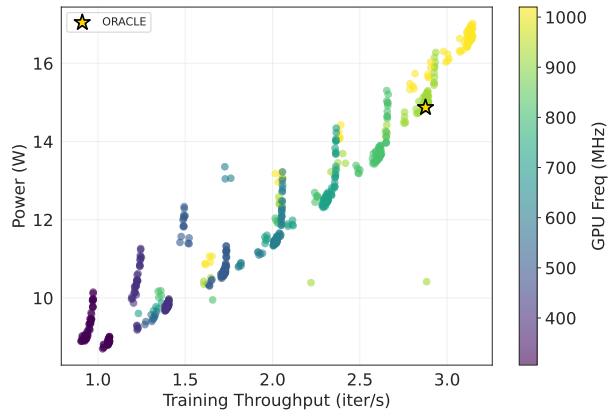
Figure 4.10: Sat-NeRF: Pareto frontier of the throughput–power trade-off (ORACLE starred).



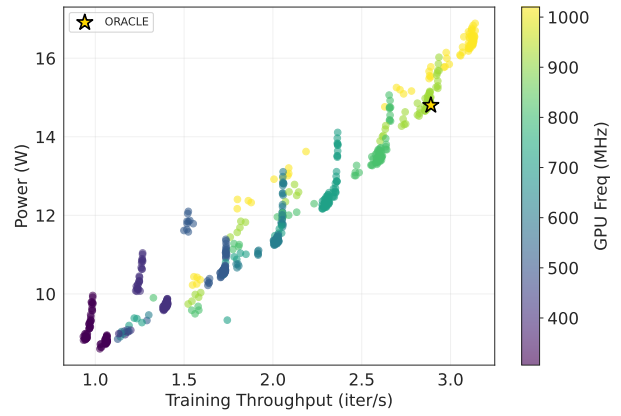
(a) JAX_214



(b) JAX_260

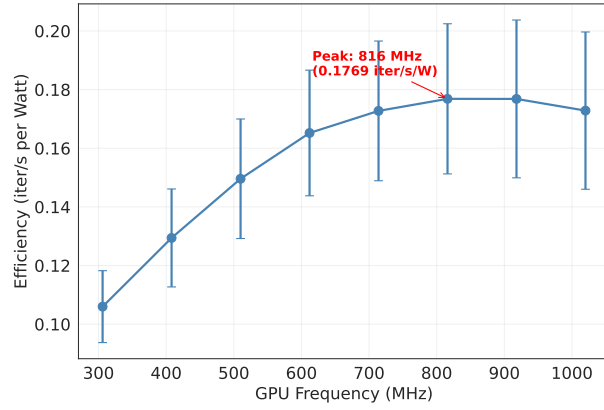


(c) JAX_068

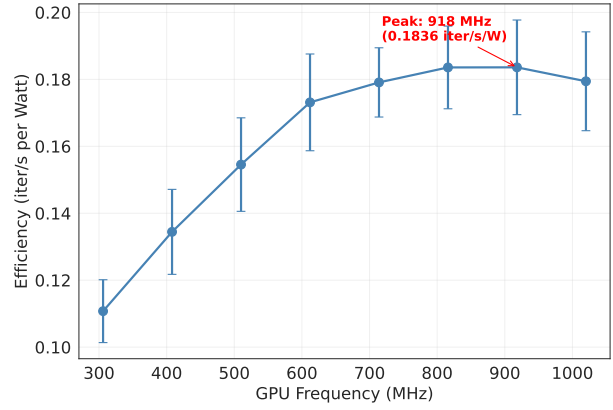


(d) JAX_004

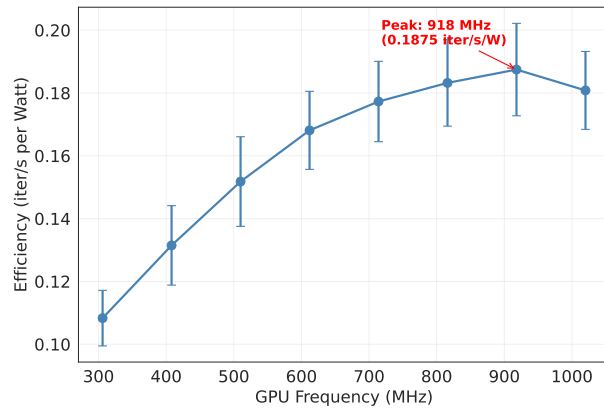
Figure 4.11: S-NeRF: throughput versus power for all profiled configurations, colored by GPU frequency (ORACLE starred).



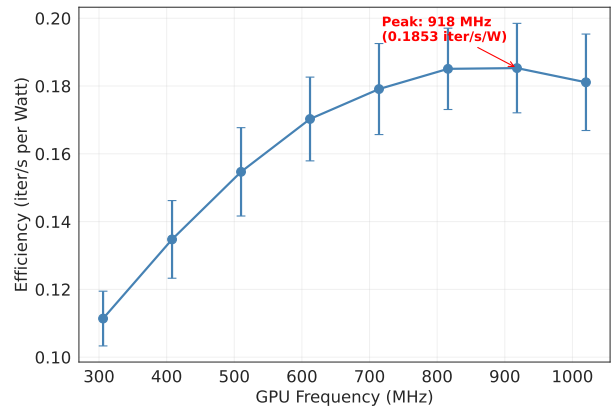
(a) JAX_214



(b) JAX_260

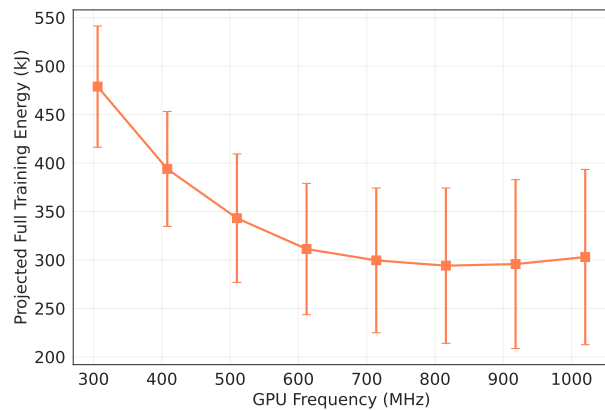


(c) JAX_068

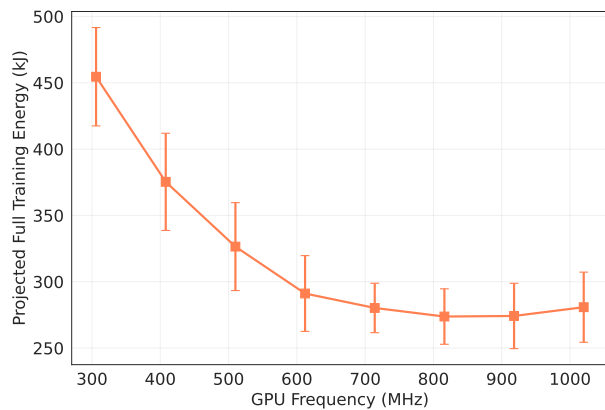


(d) JAX_004

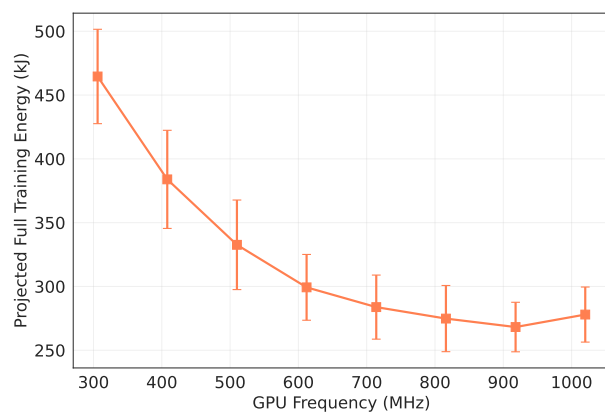
Figure 4.12: S-NeRF: energy efficiency (iter/s per watt) versus GPU frequency; efficiency peaks at an intermediate frequency.



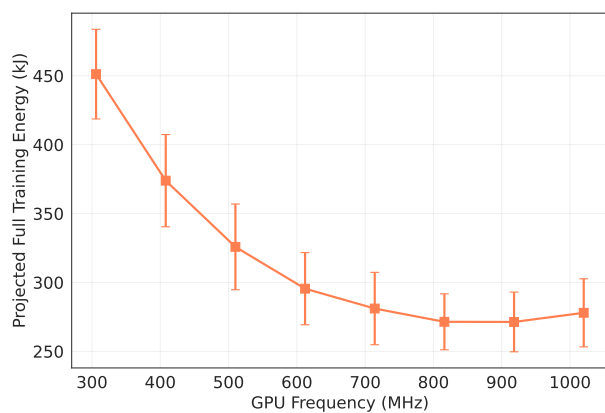
(a) JAX_214



(b) JAX_260



(c) JAX_068



(d) JAX_004

Figure 4.13: S-NeRF: projected full-training energy (50,000 steps) versus GPU frequency; energy is minimized at the intermediate efficiency-peak frequency.

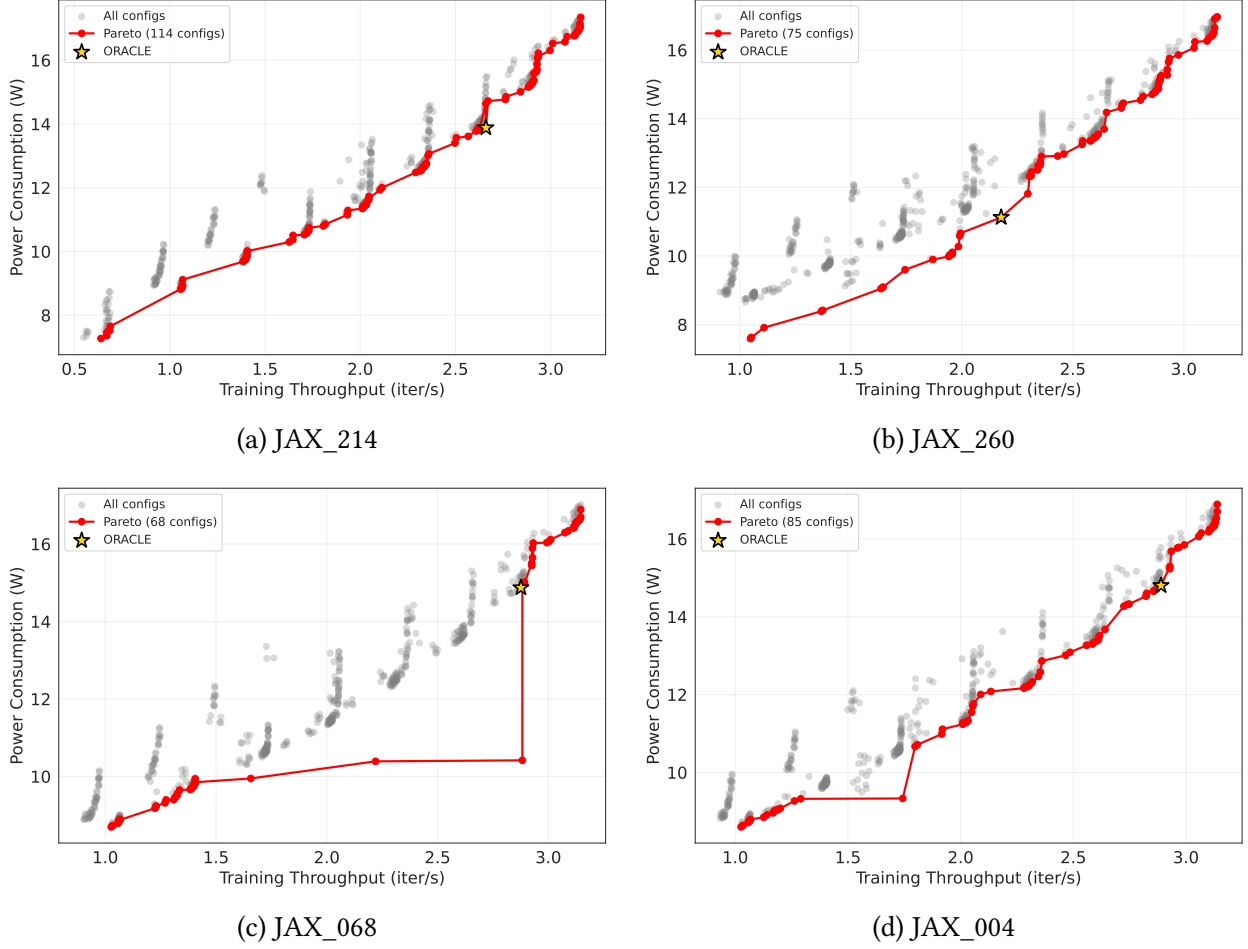


Figure 4.14: S-NeRF: Pareto frontier of the throughput–power trade-off (ORACLE starred).

4.3 ORACLE Configurations

Tables 4.6 and 4.7 list the ORACLE for each scene, per workload. The same structure holds for both. First, every ORACLE selects an intermediate GPU frequency (714–918 MHz), never the maximum 1,020 MHz. Second, the ORACLEs use different core counts (2, 3 and 4); because cores have near-zero throughput impact, these differences are noise: any core count yields essentially the same performance at the same CPU/GPU frequency. Third, the ORACLE CPU frequency is moderate to high in every case, indicating that a moderately high CPU frequency is sufficient.

Table 4.6: ORACLE (highest-efficiency feasible) configuration per scene (Sat-NeRF, target 3.0 iter/s).

Scene	Cores	CPU (MHz)	GPU (MHz)	iter/s	Power (W)	Efficiency
JAX_214	2	1526.4	918	3.376	18.11	0.1864
JAX_260	3	1425.6	816	3.065	16.67	0.1839
JAX_068	4	1526.4	918	3.375	18.07	0.1868
JAX_004	2	1526.4	918	3.428	18.01	0.1903

Table 4.7: ORACLE (highest-efficiency feasible) configuration per scene (S-NeRF, target 1.0 iter/s).

Scene	Cores	CPU (MHz)	GPU (MHz)	iter/s	Power (W)	Efficiency
JAX_214	3	1224.0	816	2.660	13.88	0.1916
JAX_260	3	619.2	714	2.177	11.13	0.1957
JAX_068	3	1022.4	918	2.876	14.87	0.1934
JAX_004	2	1324.8	918	2.889	14.80	0.1952

4.3.1 Cross-Scene Transferability

A practical question is how well one scene’s ORACLE performs on another. Tables 4.8 and 4.9 report the efficiency rank of each ORACLE when applied to every scene, for both workloads. For Sat-NeRF transfer is strong in every case (worst cross-scene rank 28 out of 600, top 4.7%). For S-NeRF it is likewise strong for three of the four scene ORACLEs (worst rank 44/600); the JAX_260 S-NeRF ORACLE is the exception, transferring poorly because its efficiency peak is a shallow, scene-specific optimum. An ORACLE found on one scene is therefore generally a strong starting point for another scene without re-profiling, a property the optimizer’s search-space reduction in Section 4.6 exploits.

Table 4.8: Cross-scene transferability: efficiency rank (out of 600) of each scene’s ORACLE when evaluated on every scene (Sat-NeRF). Diagonal (native) entries are rank 1 by definition.

ORACLE from	on 214	on 260	on 068	on 004
JAX_214	1	4	10	1
JAX_260	16	1	12	11
JAX_068	4	28	1	8
JAX_004	1	4	10	1

Table 4.9: Cross-scene transferability: efficiency rank (out of 600) of each scene’s ORACLE when evaluated on every scene (S-NeRF). Diagonal (native) entries are rank 1 by definition.

ORACLE from	on 214	on 260	on 068	on 004
JAX_214	1	17	44	38
JAX_260	141	1	178	161
JAX_068	6	5	1	7
JAX_004	8	97	13	1

4.4 Thermal and Accuracy Characteristics

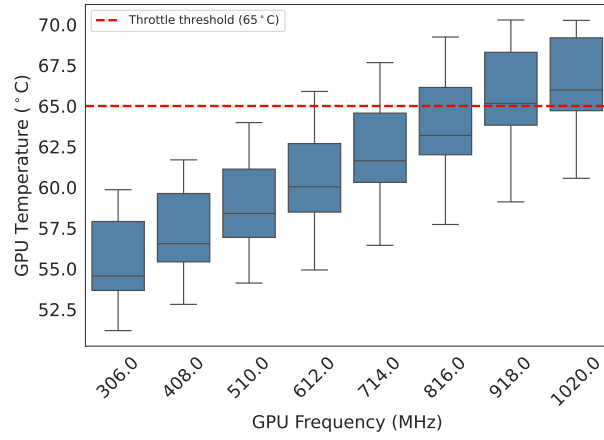
Tables 4.10 and 4.11 summarize per-scene thermal risk and reconstruction accuracy for both workloads. Thermal risk (the fraction of configurations whose GPU temperature exceeds 65°C) is scene-dependent and markedly lower for S-NeRF (0–8%) than for Sat-NeRF (18–33%), reflecting S-NeRF’s lower board power. Reconstruction accuracy (PSNR/MAE) is constant within each scene across all configurations for both workloads, confirming that hardware configuration affects only training speed and energy cost, not model quality; the two workloads reach nearly identical accuracy on each scene, since they train the same scenes for the same budget. Accuracy varies between scenes with imagery difficulty: JAX_004 is the easiest to reconstruct (~ 23.1 dB) and JAX_214 the hardest (~ 19.3 dB). Thermal behavior is illustrated in Figures 4.15–4.20.

Table 4.10: Per-scene efficiency peak, thermal risk (configs exceeding 65°C), and reconstruction accuracy (Sat-NeRF).

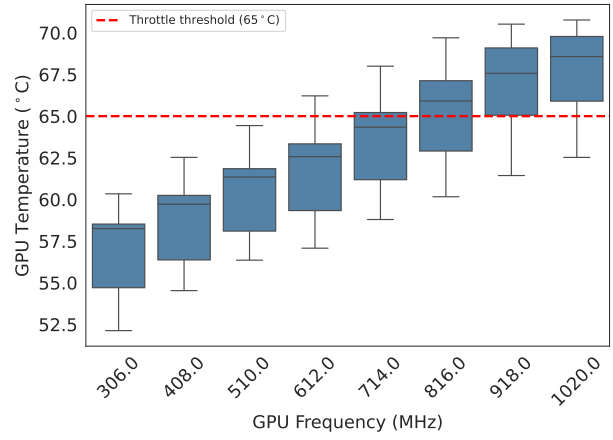
Scene	Eff. peak GPU (MHz)	Thermal risk (%)	PSNR (dB)	MAE (m)
JAX_214	918	23	19.29	6.92
JAX_260	816	33	20.89	4.10
JAX_068	918	27	20.05	3.64
JAX_004	918	18	23.09	2.67

Table 4.11: Per-scene efficiency peak, thermal risk (configs exceeding 65°C), and reconstruction accuracy (S-NeRF).

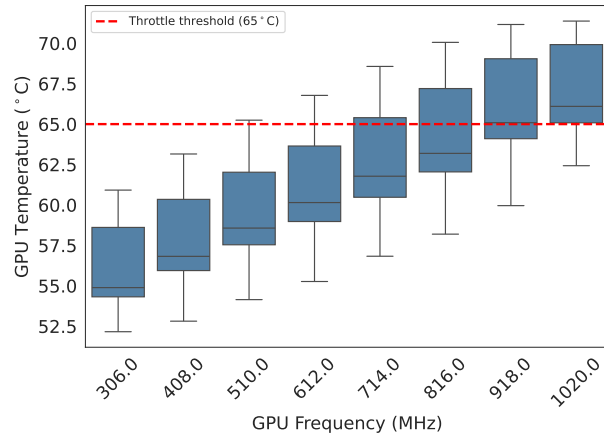
Scene	Eff. peak GPU (MHz)	Thermal risk (%)	PSNR (dB)	MAE (m)
JAX_214	816	8	19.26	6.92
JAX_260	918	0	20.97	4.20
JAX_068	918	2	20.09	3.69
JAX_004	918	2	23.12	2.70



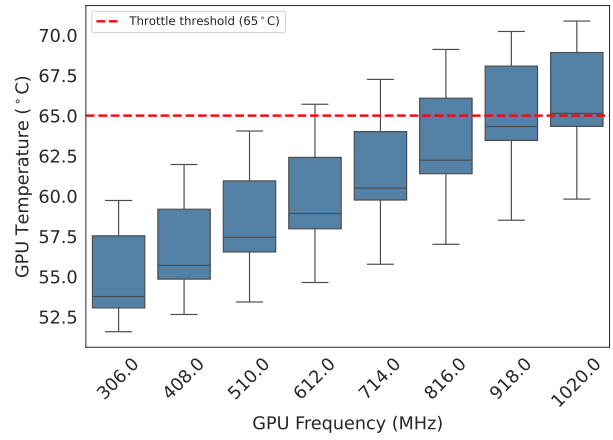
(a) JAX_214



(b) JAX_260

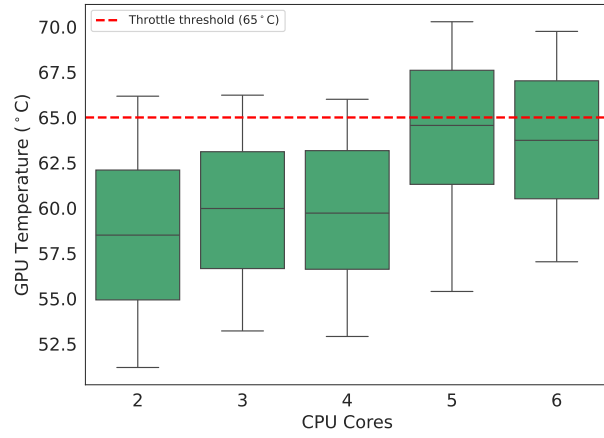


(c) JAX_068

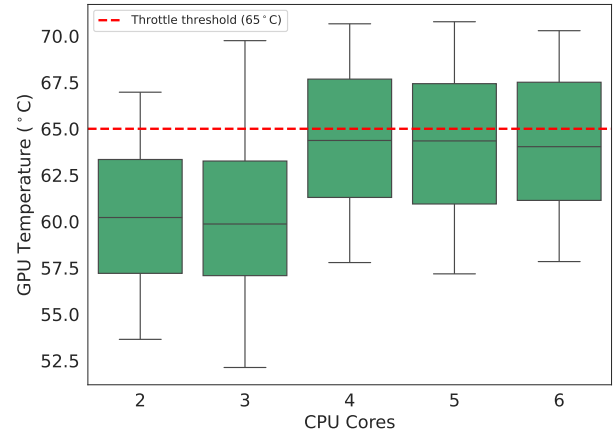


(d) JAX_004

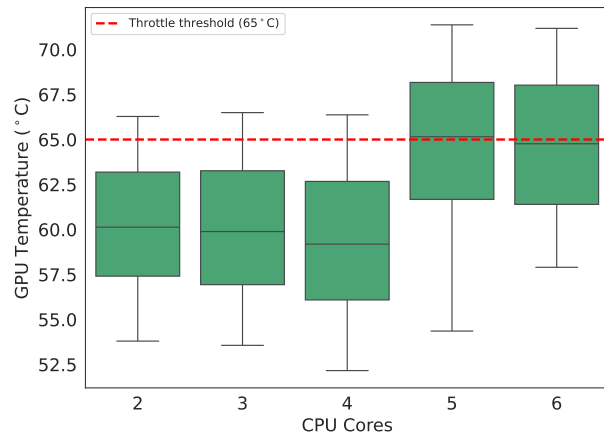
Figure 4.15: Sat-NeRF: GPU temperature versus GPU frequency.



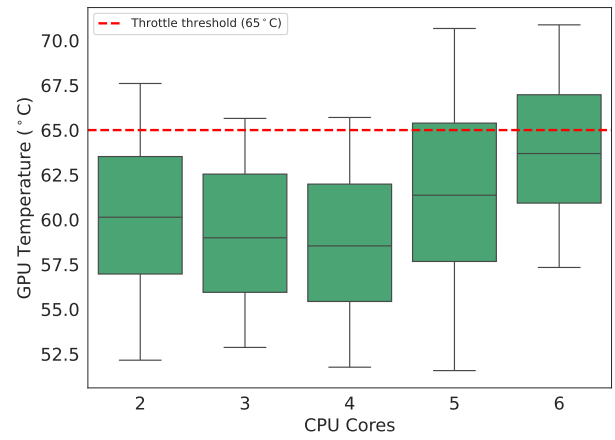
(a) JAX_214



(b) JAX_260

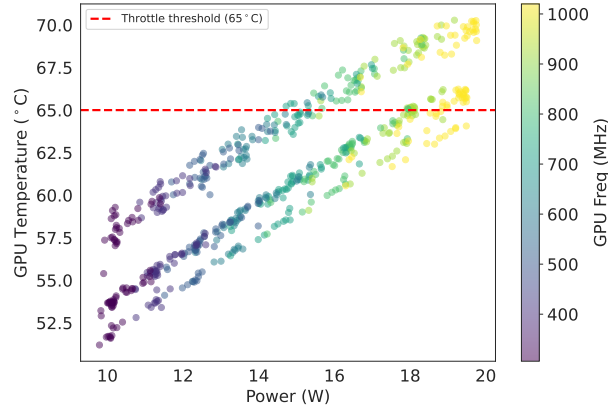


(c) JAX_068

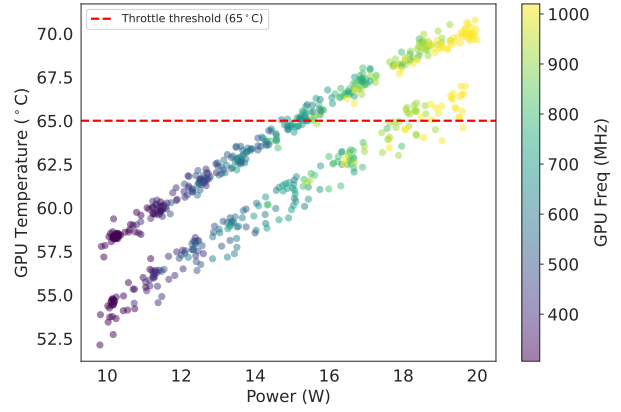


(d) JAX_004

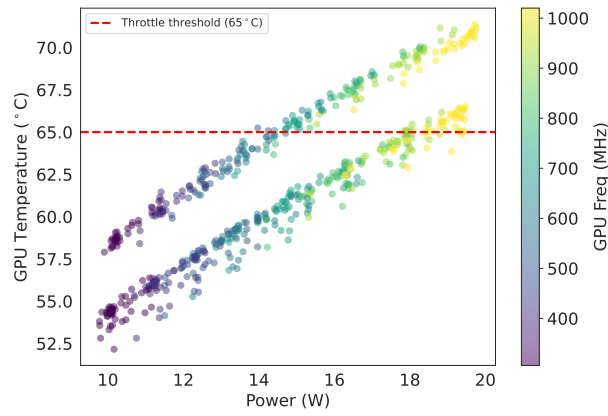
Figure 4.16: Sat-NeRF: GPU temperature versus CPU core count.



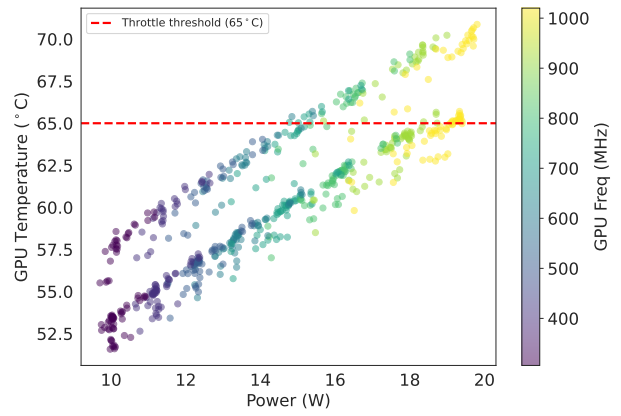
(a) JAX_214



(b) JAX_260

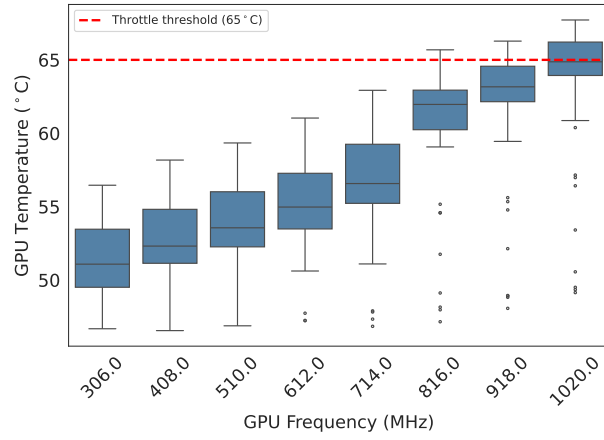


(c) JAX_068

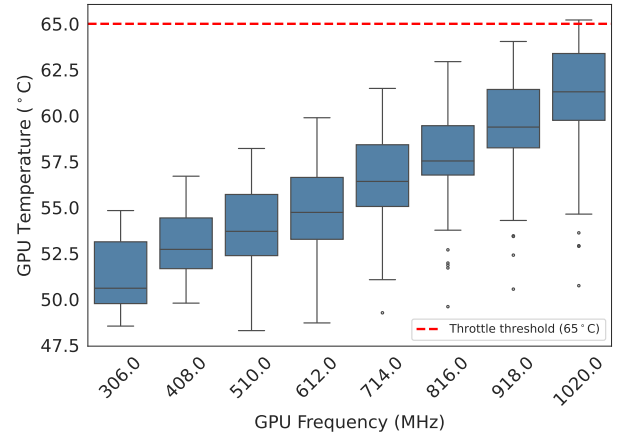


(d) JAX_004

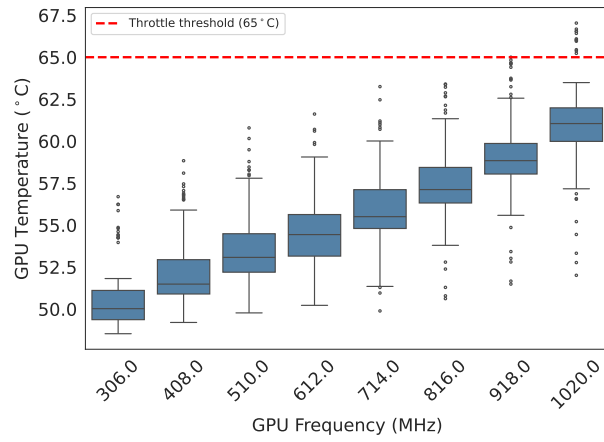
Figure 4.17: Sat-NeRF: GPU temperature versus power, colored by GPU frequency.



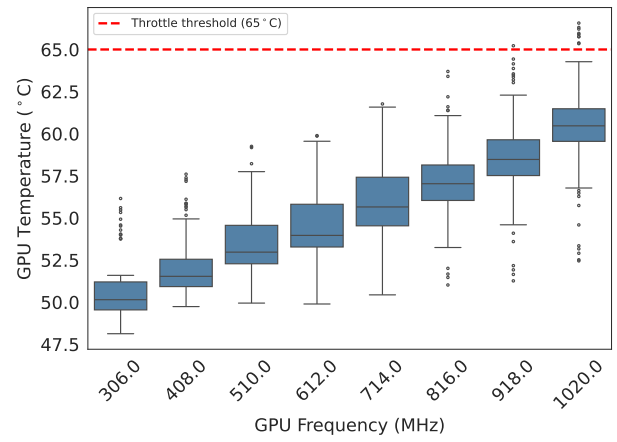
(a) JAX_214



(b) JAX_260

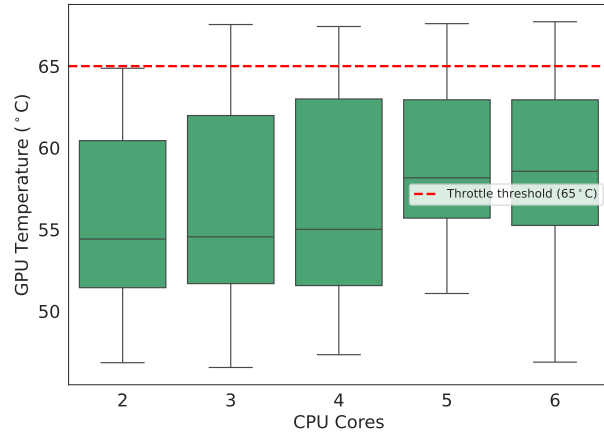


(c) JAX_068

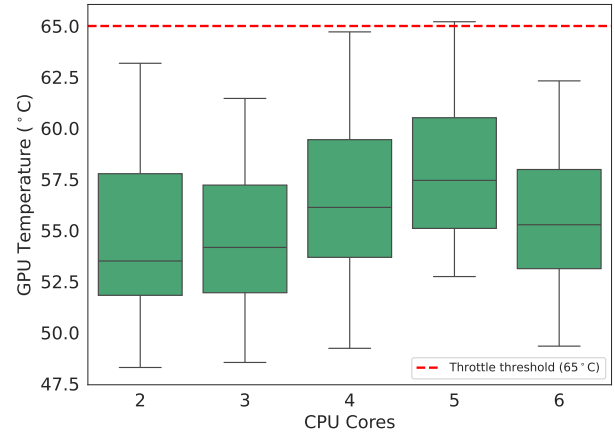


(d) JAX_004

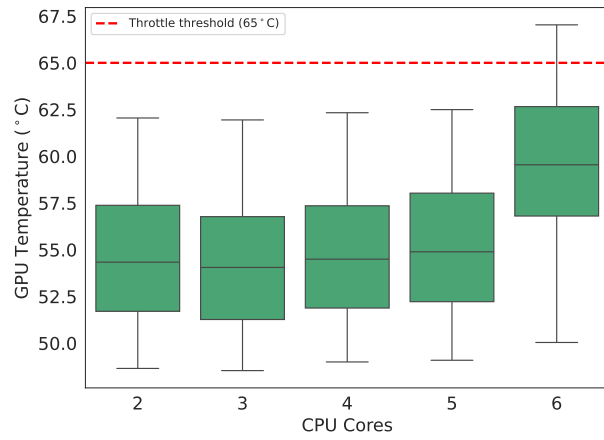
Figure 4.18: S-NeRF: GPU temperature versus GPU frequency.



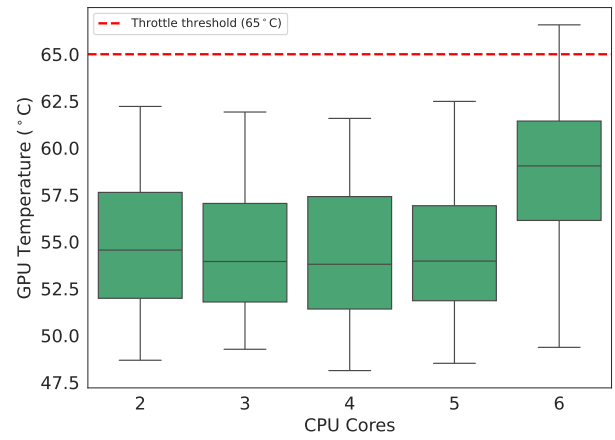
(a) JAX_214



(b) JAX_260



(c) JAX_068



(d) JAX_004

Figure 4.19: S-NeRF: GPU temperature versus CPU core count.

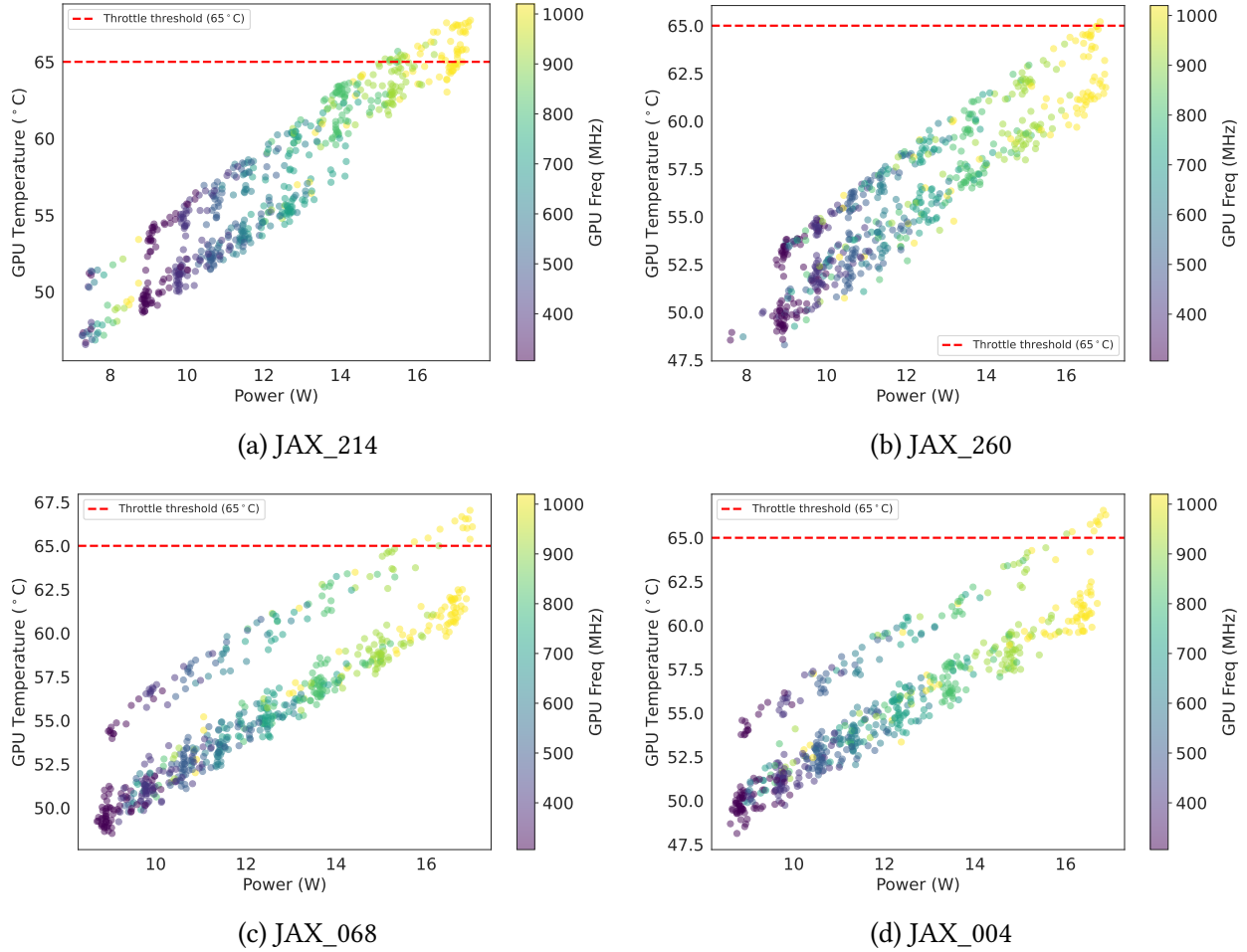


Figure 4.20: S-NeRF: GPU temperature versus power, colored by GPU frequency.

4.5 Optimizer Validation

Exhaustive profiling is prohibitively expensive in practice (hundreds of training runs per scene). The optimizer instead uses Latin Hypercube Sampling for initial exploration followed by correlation-guided exploitation to converge toward high-efficiency regions. We ran the optimizer on all four scenes, on the full 600-configuration space, for both workloads, to test whether the structural insights above can be discovered efficiently and whether they hold for both workloads.

4.5.1 Optimizer versus ORACLE

Tables 4.12 and 4.13 compare the optimizer’s best configuration against each scene’s ORACLE. To avoid crediting the optimizer with favorable measurement noise, we rank the configuration it *selected* by that configuration’s own profiled efficiency, rather than by the (noisier) value observed during the search; the gap is likewise the efficiency shortfall of the selected configuration relative to the ORACLE. On Sat-NeRF the optimizer lands within the top few percent on every scene using only a handful of episodes: it selected the *exact* ORACLE on JAX_004 (rank 1/600), and reached ranks 10–38 out of 600 on the others, with efficiency gaps below 1.6%. Every run independently minimized CPU cores to 2 and converged to a GPU frequency at or adjacent to its scene’s efficiency peak (816–918 MHz), rediscovering the profiling insights with no prior knowledge. Figure 4.21 show the search trajectories for the four Sat-NeRF scenes.

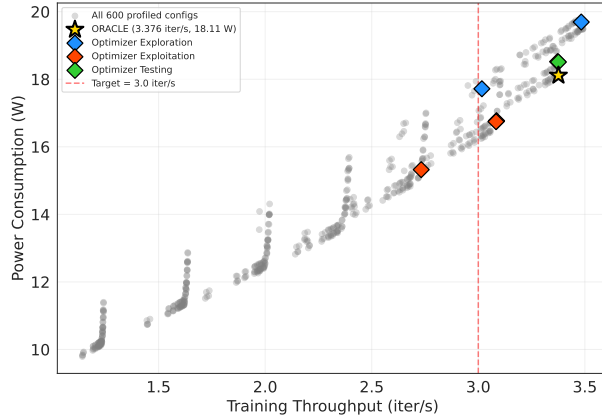
The S-NeRF results in Table 4.13 show the same behavior on the second workload: the optimizer again converges to the low-core, 816–918 MHz efficiency region and lands within **0.8–4.5%** of the S-NeRF ORACLE efficiency. Here the efficiency gap, rather than the rank, is the meaningful metric, because the S-NeRF efficiency landscape is markedly flat near its peak: many configurations lie within a few percent of the maximum, so a small efficiency difference maps to a large rank difference (for example, JAX_214’s 1.1% gap is rank 13/560). The one weaker case, JAX_260 (4.5%), is the only scene where the optimizer settled on 816 MHz while that scene’s S-NeRF efficiency peak lies elsewhere. Figure 4.22 show the S-NeRF search trajectories overlaid on the profiling.

Table 4.12: The optimizer’s best configuration versus the ORACLE (Sat-NeRF, target 3.0 iter/s). Configurations are cores/CPU MHz/GPU MHz; the optimizer’s efficiency and gap are those of the selected configuration evaluated in the profiling, so the gap is the true efficiency shortfall to the ORACLE, and rank is that configuration’s efficiency rank against the profiled distribution.

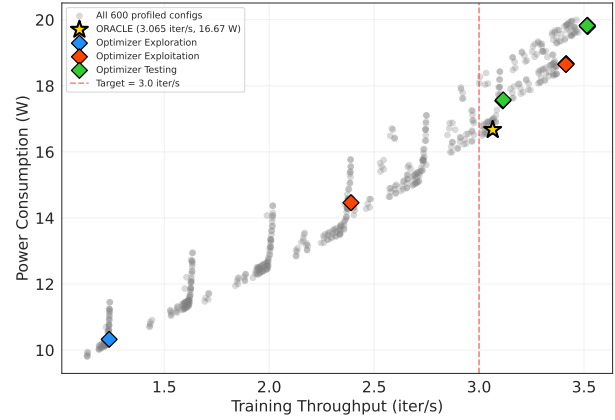
Scene	ORACLE cfg	ORACLE eff.	Optimizer cfg	Optimizer eff.	Gap	Rank
JAX_214	2c/1526.4/918	0.1864	2c/1526.4/816	0.1840	1.29%	38/600
JAX_260	3c/1425.6/816	0.1839	2c/1627.2/918	0.1811	1.51%	31/600
JAX_068	4c/1526.4/918	0.1868	2c/1526.4/918	0.1864	0.25%	10/600
JAX_004	2c/1526.4/918	0.1903	2c/1526.4/918	0.1903	0.00%	1/600

Table 4.13: The optimizer’s best configuration versus the ORACLE (S-NeRF, target 1.0 iter/s). Columns as in Table 4.12; ranks are over the 560 configurations above the minimum CPU frequency. Because the S-NeRF efficiency landscape is flat near its peak, the efficiency gap is a more meaningful measure of quality than the rank.

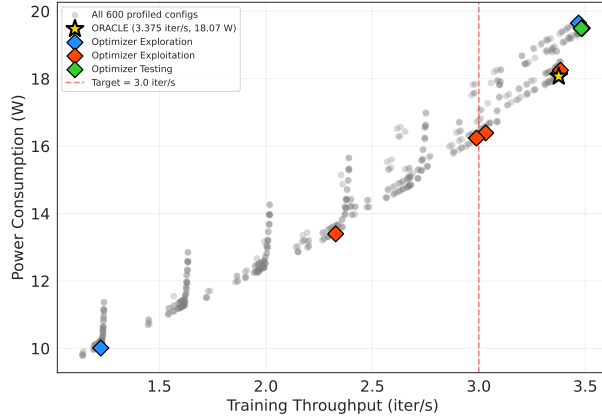
Scene	ORACLE cfg	ORACLE eff.	Optimizer cfg	Optimizer eff.	Gap	Rank
JAX_214	3c/1224.0/816	0.1916	2c/1526.4/918	0.1895	1.1%	13/560
JAX_260	3c/ 619.2/714	0.1957	2c/1526.4/816	0.1870	4.5%	152/560
JAX_068	3c/1022.4/918	0.1934	2c/1627.2/918	0.1894	2.1%	75/560
JAX_004	2c/1324.8/918	0.1952	2c/1425.6/816	0.1936	0.8%	29/560



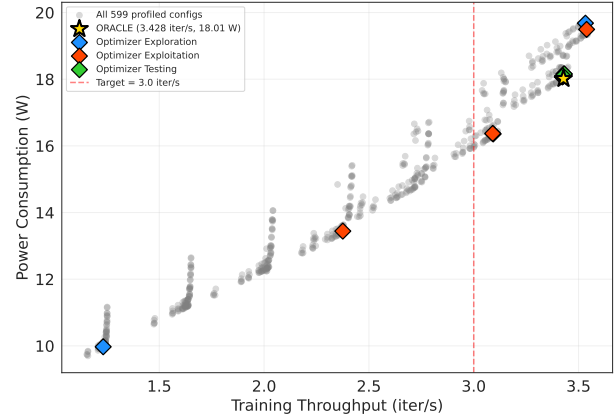
(a) JAX_214



(b) JAX_260



(c) JAX_068



(d) JAX_004

Figure 4.21: Sat-NeRF: optimizer search trajectory overlaid on all profiled configurations, colored by phase (ORACLE starred).

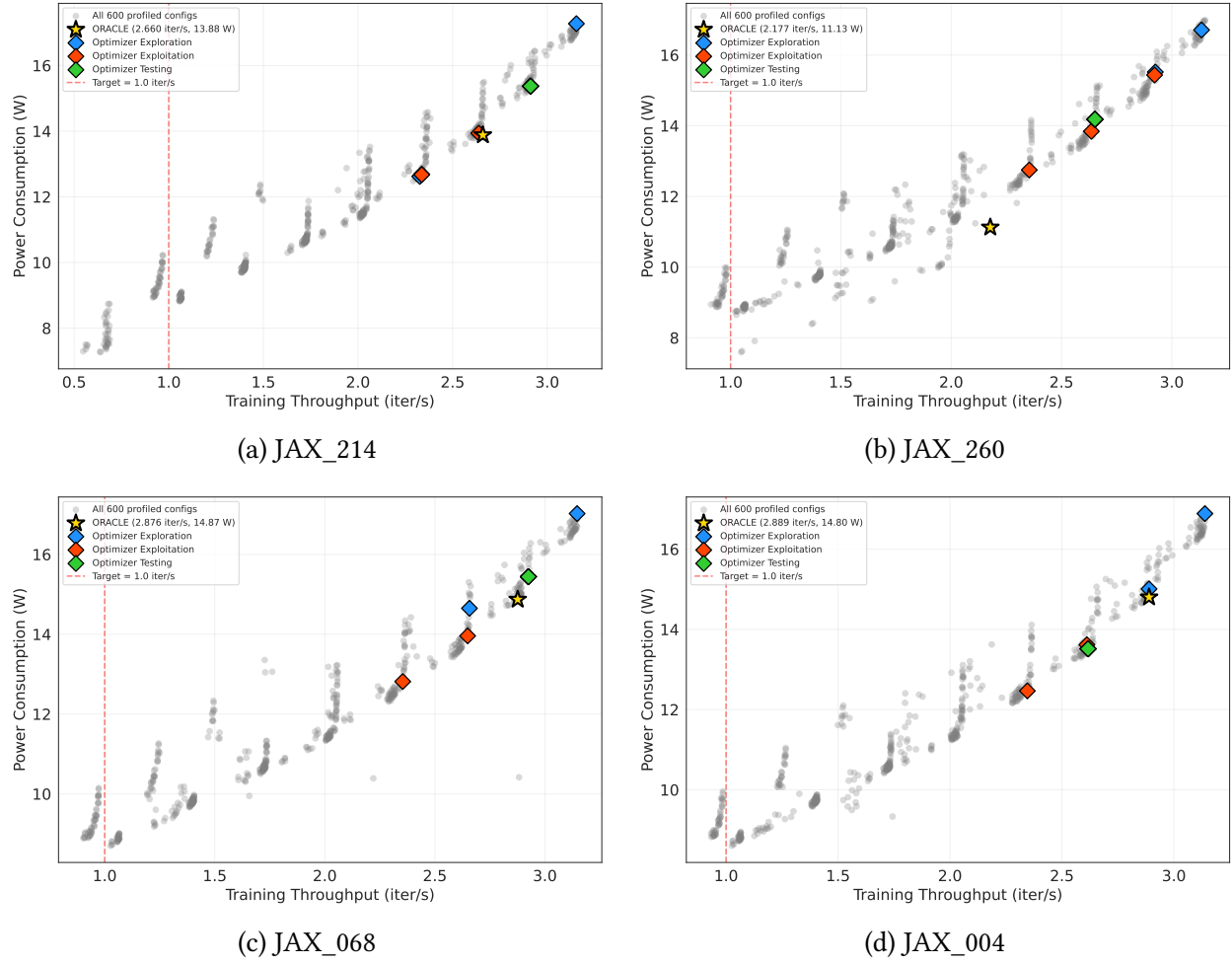


Figure 4.22: S-NeRF: optimizer search trajectory overlaid on the profiling, colored by phase (ORACLE starred).

4.5.2 Search Efficiency

Tables 4.14 and 4.15 quantify the optimizer’s search cost. Each run explored only 5–6 unique configurations (roughly 1% of the 600-configuration space). Sat-NeRF runs consumed 2.1–3.1 hours of device time and S-NeRF runs 3.2–3.8 hours (the difference is the S-NeRF per-run preprocessing pass), versus the tens of hours required to profile a scene exhaustively, a reduction of more than an order of magnitude. On Sat-NeRF the optimizer reached the top few percent of the search space on every scene, selecting the *exact* ORACLE on JAX_004; on S-NeRF it reached within a few percent of the ORACLE efficiency on all four scenes.

Table 4.14: Optimizer search efficiency per scene (Sat-NeRF). The optimizer evaluates $\sim 1\%$ of the space yet reaches the top few percent.

Scene	Search space	Episodes	Unique cfigs	Device time (h)	Best rank
JAX_214	600	8	5	2.12	38/600
JAX_260	600	11	5	2.58	31/600
JAX_068	600	12	6	2.82	10/600
JAX_004	600	11	6	3.10	1/600

Table 4.15: Optimizer search efficiency per scene (S-NeRF). The optimizer evaluates $\sim 1\%$ of the space yet reaches within a few percent of the ORACLE efficiency; best ranks are over the 560 configurations above the minimum CPU frequency.

Scene	Search space	Episodes	Unique cfigs	Device time (h)	Best rank
JAX_214	600	13	6	3.77	13/560
JAX_260	600	13	6	3.70	152/560
JAX_068	600	13	5	3.24	75/560
JAX_004	600	12	5	3.42	29/560

4.6 Search Space Reduction

The exhaustive profiling reveals that large regions of the 600-configuration space can never produce competitive results, motivating a reduced search space for future optimizer runs. Two pruning rules follow directly from the landscape analysis. First, the five lowest GPU frequencies (306–714 MHz) never reach the throughput target and can be excluded, leaving only $\{816, 918, 1020\}$ MHz. Second, because CPU cores have no throughput impact, restricting cores to $\{2, 3\}$ loses nothing while halving that dimension; combined with a CPU-frequency floor of 518.4 MHz, this yields the recommended space in Table 4.16.

Table 4.16: Recommended reduced search space, derived from the profiling landscape. This yields 78 configurations, an 87% reduction from the full space.

Hardware parameter	Recommended range	Rationale
CPU cores	$\{2, 3\}$	Zero throughput impact; minimize for power
CPU frequency	≥ 518.4 MHz (13 levels)	Lowest levels never meet the target
GPU frequency	$\{816, 918, 1020\}$ MHz	Only these reach the target; 918 is the peak

This reduction is derived from the Sat-NeRF landscape (target 3.0 iter/s) and validated by it: every Sat-NeRF ORACLE’s frequency settings fall within the reduced space, and the S-NeRF efficiency peaks likewise fall at 816–918 MHz, so the same reduced GPU set suits both workloads. Where an ORACLE uses more than two cores its two-core equivalent is near-identical since cores carry no throughput impact. Restricting future searches to this space would raise the optimizer’s coverage density more than eightfold, since a handful of unique configurations cover a far larger fraction of 78 than of 600 configurations, sharply increasing the probability of sampling near-optimal configurations during exploration. This matters because, even on the *full* space, the optimizer already reached the top few percent on every Sat-NeRF scene and within a few percent of the ORACLE efficiency on every S-NeRF scene (Section 4.5); the reduction is thus a cost lever rather than a correctness requirement. We recommend it for deployment while noting that the optimizer remains effective without it.

4.7 Summary

Exhaustive profiling of 4,800 configuration measurements across two workloads and four scenes establishes a consistent structure: GPU frequency dominates the throughput–power operating point, CPU cores are irrelevant to throughput, and energy efficiency peaks at an intermediate GPU frequency (816 or 918 MHz depending on the scene) rather than the maximum. Each scene’s ORACLE reflects this structure, and ORACLEs transfer well across scenes. The optimizer rediscovers these insights automatically, reaching the top few percent of the search space (and the exact ORACLE on Sat-NeRF JAX_004) while evaluating only about 1% of the space and using an order of magnitude less device time than exhaustive search. The same structure and the same optimizer behavior hold for both workloads: on the second workload, S-NeRF, the efficiency landscape is again GPU-dominated with an intermediate-frequency peak, and the optimizer lands within 0.8–4.5% of the S-NeRF ORACLE efficiency on all four scenes. A profiling-informed reduction of the search space to 78 configurations further accelerates convergence without excluding any near-optimal configuration.

CHAPTER 5

DISCUSSION AND RELATED WORKS

This chapter interprets the results of Chapter 4, discusses what they imply for edge NeRF training and for the design of the optimizer, states the limitations of the study, and positions the work within the related literature.

5.1 Interpreting the Configuration Landscape

The exhaustive profiling in Section 4.2 yields a consistent and counter-intuitive picture. Throughput and power on the Jetson Orin Nano are governed almost entirely by a single hardware parameter, the GPU frequency (Table 4.2), while the CPU core count is statistically indistinguishable from noise and the CPU frequency contributes only weakly. Two consequences follow.

First, the effective dimensionality of the problem is far smaller than the nominal three-parameter, 600-configuration space suggests. Much of the apparent search space is redundant: the more than two hundred configurations that differ only in core count trace out essentially the same operating point. This is why a search that identifies and prioritizes the influential hardware parameter can succeed with so few evaluations, and it is the structural fact that the search-space reduction in Section 4.6 exploits.

Second, and more consequential for deployment, the most energy-efficient configuration is never the fastest one. Energy efficiency peaks at an intermediate GPU frequency (816 or 918 MHz) and falls at the 1,020 MHz maximum (Figure 4.8), and the projected full-training energy is

minimized at that same intermediate point (Figure 4.9). A manufacturer “maximum-performance” preset, or the common practice of pinning `jetson_clocks` to the ceiling, therefore trades a large increase in power and heat for a marginal throughput gain, and in fact spends more energy on the same training run. The thermal results reinforce this: the highest GPU frequency also produces the highest temperatures and the largest fraction of thermally-at-risk configurations (Figure 4.15, Table 4.10). For a power- and thermally-constrained edge platform, the efficient operating point is an interior point of the space, exactly the kind of point a coarse preset cannot express.

A third observation concerns generalization. The hardware performance envelope is nearly scene-invariant (Table 4.4), and each scene’s ORACLE transfers to the others within the top few percent (Table 4.8). This decoupling (the hardware behavior stays stable while only the efficiency-peak frequency shifts slightly by scene) is what makes a lightweight, transferable optimizer viable: the search need not be repeated from scratch for every new scene.

5.2 On the Behavior of the Optimizer

The optimizer rediscovers the structure above with no prior knowledge of it. Guided only by distance-covariance weights estimated online, every run independently drove the CPU cores to their minimum and selected the GPU frequency at the scene’s efficiency peak (Section 4.5), converging to within the top few percent of the space (and to the exact ORACLE, rank 1/600, on JAX_004), in roughly ten iterations while evaluating about 1% of the configurations (Tables 4.12 and 4.14). Three design choices explain this behavior.

The dependence-guided step (Section 3.6) is the core mechanism. Because the step size for each hardware parameter is scaled by its distance correlation with throughput and power, the search spends its budget almost entirely on the GPU frequency and leaves the inert core-count hardware parameter at its power-minimizing setting. Crucially, this pinning is derived from the measured correlation structure ($\gamma_i \approx 0$) rather than hardcoded (step 8 in Figure 3.2); the same rule would pin a different hardware parameter on a workload where a different parameter is inert. That the inert hardware parameter identified automatically (CPU cores) matches the one identified

by exhaustive profiling (Table 4.2) is direct evidence that the dependence measure is doing the intended work.

The feasibility-aware efficiency score (Section 3.4) shapes the search toward the constrained objective. By ranking every infeasible configuration below every feasible one and penalizing settings that draw power without meeting the target, the efficiency score steers the search along the throughput–power frontier toward the knee where the target is met at the least power (Figure 4.10) rather than toward raw speed.

Finally, the profiling-informed search-space reduction (Section 4.6) and the optimizer are complementary. The reduction removes regions that provably cannot meet the target, which raises the coverage density of the exploration phase. Even on the full space, however, the optimizer already reached the top few percent on every scene and the exact ORACLE on JAX_004, so the reduction accelerates convergence without being necessary for its success.

5.3 Practical Implications

Taken together, the findings translate into concrete guidance for Sat-NeRF training on the Jetson platform:

- **Do not run at the maximum GPU frequency.** Prefer the efficiency-peak frequency (816–918 MHz), which minimizes energy per training run and lowers thermal risk at negligible cost in throughput.
- **Minimize CPU cores.** Because cores have no measurable effect on throughput, running the fewest cores that keep the system responsive saves power for free.
- **Use a moderate CPU frequency.** The ORACLEs cluster around 1.3–1.5 GHz; higher CPU frequencies add power without improving throughput.
- **Warm-start from a transferred configuration.** Since ORACLEs transfer well across scenes, a known-good configuration from one scene is a strong starting point that the optimizer can then refine online.

- **Adopt the reduced search space by default.** The 78-configuration space of Table 4.16 excludes only configurations that provably cannot meet the target and measurably speeds convergence.

5.4 Limitations

Several limitations bound the scope of these results and motivate future work.

Single device and workload family. All experiments use one device (the Jetson Orin Nano 8GB) and two closely related NeRF training workloads (Sat-NeRF and S-NeRF). The two workloads give consistent results (GPU frequency dominates and the optimizer reaches near-ORACLE efficiency on both), which strengthens the findings within this family, but both are GPU-bound satellite-NeRF training; a CPU-bound or memory-bound workload would shift the influential hardware parameter. The optimizer is designed to adapt to such a shift because its hardware parameter weighting is measured rather than assumed, but this adaptivity has not yet been tested on other devices or on structurally different workloads.

Reduced hardware parameter set. The configuration space fixes the memory frequency at its maximum and omits the concurrency level, which is not meaningful for a single-instance training job. Reintroducing these hardware parameters would enlarge the space and could introduce interactions not observed here.

Single throughput target. Results are reported for one target ($\tau_{\text{target}} = 3.0$ iter/s). A different target moves the feasible frontier and hence the optimum; a sweep over targets would map how the recommended configuration changes with the performance requirement.

Component attribution. The relative contribution of the dependence-guided hardware parameter pinning versus the underlying guided search has not been isolated. An ablation that selectively disables individual heuristics would quantify how much each component contributes to the final result.

5.5 Related Work

5.5.1 Efficient Neural Radiance Fields

A large line of work accelerates NeRF by changing the model or its representation: multiresolution hash-grid encodings that cut training time by orders of magnitude (Instant-NGP [10]), depth supervision that reduces the number of views and iterations needed (DS-NeRF [1], SparseSat-NeRF [21]), and baked or real-time variants for fast inference. These methods reduce the algorithmic cost of training or rendering and are orthogonal and complementary to this thesis: they change how many operations training requires, whereas we optimize the hardware operating point at which those operations run. The optimizer would apply unchanged on top of a faster NeRF variant.

5.5.2 Energy-Efficient Deep Learning on Edge Devices

A substantial body of work tunes DVFS and related hardware parameters for deep-learning energy efficiency. ALERT [18] jointly selects DVFS settings and model variants to meet latency and accuracy targets, but depends on offline profiling that must be repeated per deployment. DVFO [22] learns a DVFS policy by deep reinforcement learning for edge-cloud collaborative inference, and PolyThrottle [19] frames energy-optimal inference serving as constrained Bayesian optimization. Zeus [20] optimizes the GPU power-limit and batch-size trade-off for DNN *training* energy in the data center, and PowerTrain [14] builds transferable time and power predictors for DNN training on Jetson Orin hardware. Two gaps distinguish our setting. First, almost all of this work targets *inference*, a short and repeatable workload in which a large tuning or training budget is amortized over many invocations; NeRF training is long and effectively single-shot per scene, so the search budget itself must be small. Second, the methods that reach near-optimal results do so through offline profiling or learned predictors that must be constructed per model or per device, whereas the optimizer requires neither: it discovers the influential hardware parameters online from a handful of measurements taken during the very run it is optimizing.

5.5.3 Configuration Tuning and Hardware Parameter Importance

Deciding which hardware parameters matter is a recurring theme in systems auto-tuning. Otter-Tune [17] uses Lasso regression to rank database hardware parameters before tuning them, and Kanellis et al. [5] show that pre-selecting a small set of important hardware parameters sharply accelerates database configuration tuning. The optimizer shares this philosophy of reducing effective dimensionality by identifying influential parameters, but differs in both method and setting. It uses a non-linear dependence measure (distance covariance) computed online *during* the same search that exploits it, rather than a separate offline importance-ranking pass, and it applies to hardware DVFS parameters rather than software configuration parameters. A linear importance score such as Lasso could also miss the non-linear, interacting relationship between frequency and performance that distance covariance is able to capture.

5.5.4 Dependence- and Sensitivity-Guided Optimization

High-dimensional black-box optimization commonly exploits latent low-dimensional structure. TuRBO [4] restricts Bayesian optimization to adaptive trust regions, and SAASBO [3] places sparsity-inducing priors so that only a few dimensions dominate the surrogate. Closer to our approach, sensitivity measures have been used to select variables during optimization: Spagnol et al. [15] employ the Hilbert-Schmidt Independence Criterion to guide variable selection within Bayesian optimization. Multi-fidelity hyperparameter methods such as Hyperband [6] instead conserve budget by early-stopping unpromising configurations. The optimizer occupies a deliberately lightweight point in this design space: rather than maintaining a surrogate model, it uses the distance correlation [16] directly as a step-scaling weight, which keeps the per-iteration cost negligible relative to the training run being tuned and is well matched to a budget of roughly ten evaluations. Distance covariance detects arbitrary non-linear dependence and is zero only under independence, making it a natural fit for the non-linear, interacting hardware-performance relationship studied here.

5.5.5 Positioning

To our knowledge, no prior work characterizes the hardware-configuration landscape of NeRF training on an edge device, nor provides an online, profiling-free optimizer tailored to it. The optimizer combines a feasibility-aware objective, distance-covariance hardware parameter weighting, and a small-budget guided search to fill this gap. The evaluation in Chapter 4 shows that this combination recovers near-optimal configurations across four scenes at a fraction of the cost of exhaustive profiling, and without any offline measurement.

CHAPTER 6

CONCLUSION AND FUTURE WORK

Training neural radiance fields for satellite imagery on edge hardware places a computationally heavy, energy-hungry workload on a device with a tight power and thermal budget. The hardware configuration (the CPU and GPU frequencies and the number of active cores) strongly affects how much training progress the device makes per watt, yet the efficient operating points are hidden in a large, non-linear, interacting search space. This thesis set out to understand that space and to select good configurations within it cheaply, without the exhaustive offline profiling that prior approaches require.

6.1 Summary and Contributions

The thesis made two complementary contributions, one characterizing the problem and one solving it.

First, Chapter 4 presented an exhaustive characterization of the hardware-configuration landscape of Sat-NeRF and S-NeRF training on the NVIDIA Jetson Orin Nano 8GB, spanning 4,800 configuration measurements across four satellite scenes. The landscape has a clear and consistent structure: GPU frequency dominates both throughput and power, CPU frequency has only a weak effect, and CPU core count is statistically indistinguishable from noise. The most important practical finding is that the maximum GPU frequency is not the most efficient one. Energy efficiency, and total training energy, are optimized at an intermediate GPU frequency (816 or 918

MHz depending on the scene), so the common practice of pinning the device to its performance ceiling wastes both energy and thermal headroom. The per-scene optimal (ORACLE) configurations reflect this structure and transfer well across scenes, indicating that the hardware behavior is largely decoupled from scene content.

Second, Chapter 3 applied the online optimizer [11] that selects hardware configurations during training without any offline profiling. The optimizer uses distance covariance to measure, from a handful of live observations, how strongly each hardware parameter influences throughput and power, and concentrates its limited search budget on the hardware parameters that matter. In the evaluation it rediscovered the profiling insights automatically: every run independently minimized the CPU cores and converged onto the scene’s efficiency-peak GPU frequency, reaching the top few percent of the search space (and the exact ORACLE on JAX_004) within roughly ten iterations. In doing so it evaluated only about 1% of the configuration space and consumed an order of magnitude less device time than exhaustive profiling. Finally, the same characterization yielded a profiling-informed reduction of the search space from 600 to 78 configurations (an 87% reduction) that excludes no near-optimal configuration and measurably accelerates convergence.

Taken together, these results show that the configuration space of edge NeRF training, though large in appearance, is governed by a small number of influential hardware parameters, and that a lightweight, dependence-guided optimizer can exploit this structure to find near-optimal, energy-efficient operating points at a small fraction of the cost of exhaustive search.

6.2 Future Work

The findings and limitations of this thesis suggest several directions for future research.

Online, in-training optimization. The most immediate extension is to amortize the search into a single training run. Because per-configuration hardware throughput is stationary across training (unlike the loss), short training windows can serve as low-cost evaluation samples: a controller could adjust the CPU and GPU frequencies window by window, converge early, and then lock the best configuration for the remainder of the run. This would remove the per-scene

search overhead that remains the optimizer’s main cost, turning configuration selection into a nearly free by-product of the training the device was going to perform anyway.

Broader devices and workloads. All experiments used a single device and a single workload family. Validating the optimizer on other edge accelerators (for example the larger Jetson Orin AGX or older Xavier platforms) and on other training workloads (convolutional networks, transformers, and other NeRF variants) would test the central premise that its dependence-guided weighting adapts to whichever hardware parameter is influential. A CPU-bound or memory-bound workload, where GPU frequency no longer dominates, would be a particularly informative stress test. Reintroducing the memory-frequency and concurrency hardware parameters held fixed here would enlarge the space and probe how the optimizer scales as hardware parameter interactions grow.

Multi-objective and constraint-aware extensions. This thesis optimized energy efficiency subject to a single throughput target. Future work could sweep the target to build an explicit map from performance requirement to recommended configuration, expose the full throughput–power Pareto set to the user, and incorporate an explicit thermal constraint so that the optimizer avoids throttling-prone operating points directly rather than only implicitly through the efficiency term.

Richer dependence measures and search strategies. Distance covariance was chosen for its ability to capture non-linear dependence at negligible cost. Alternative kernel-based measures such as the Hilbert-Schmidt Independence Criterion [15], paired with a significance test, could make the pinning of inert hardware parameters more statistically principled, and surrogate-assisted or trust-region hybrids could extend the approach to substantially larger configuration spaces while preserving its small-budget behavior.

Ablation and robustness. Finally, a systematic ablation that selectively disables individual components, such as the dependence-guided hardware parameter pinning, would quantify how much each contributes to the optimizer’s performance, and a thermal-drift-aware variant that periodically re-checks the locked configuration would extend the method to very long training runs where late-run throttling can shift the optimal operating point.

6.3 Closing Remarks

As deep learning continues to move to the edge, the question of *how* to run a workload, not just *what* workload to run, becomes central to making in-situ training practical under strict power and thermal budgets. This thesis showed that for Sat-NeRF training on a Jetson device the efficient operating points are both non-obvious and discoverable: the landscape is dominated by a single hardware parameter, efficiency favors an intermediate rather than a maximal setting, and a lightweight online optimizer can find these points without costly profiling. The same principle, measuring which hardware parameters matter and spending a small budget only on those, applies well beyond this workload and offers a practical path toward energy-efficient edge training more broadly.

BIBLIOGRAPHY

- [1] K. Deng, A. Liu, J.-Y. Zhu, and D. Ramanan, “Depth-supervised nerf: Fewer views and faster training for free,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 882–12 891.
- [2] D. Derksen and D. Izzo, “Shadow neural radiance fields for multi-view satellite photogrammetry,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 1152–1161.
- [3] D. Eriksson and M. Jankowiak, “High-dimensional Bayesian optimization with sparse axis-aligned subspaces,” in *Uncertainty in Artificial Intelligence (UAI)*, 2021, pp. 493–503.
- [4] D. Eriksson, M. Pearce, J. R. Gardner, R. D. Turner, and M. Poloczek, “Scalable global optimization via local Bayesian optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [5] K. Kanellis, R. Alagappan, and S. Venkataraman, “Too many knobs to tune? towards faster database tuning by pre-selecting important knobs,” in *USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2020.
- [6] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018.
- [7] R. Marí, G. Facciolo, and T. Ehret, “Sat-nerf: Learning multi-view satellite photogrammetry with transient objects and shadow modeling using rpc cameras,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 1311–1321.

- [8] R. Marí, C. de Franchis, E. Meinhardt-Llopis, J. Anger, and G. Facciolo, “A Generic Bundle Adjustment Methodology for Indirect RPC Model Refinement of Satellite Imagery,” *Image Processing On Line*, vol. 11, pp. 344–373, 2021, <https://doi.org/10.5201/ipol.2021.352>.
- [9] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [10] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Transactions on Graphics (SIGGRAPH)*, vol. 41, no. 4, pp. 1–15, 2022.
- [11] A. N. L. Nabhaan, Z. Sukma, R. D. Rachmanto, *et al.*, “CORAL: Covariance-guided resource adaptive learning for efficient edge inference,” in *IEEE International Conference on Fog and Edge Computing (ICFEC)*, 2025.
- [12] NVIDIA, *NVIDIA Jetson Orin*, <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>, 2024.
- [13] O. Ozyesil, V. Voroninski, R. Basri, and A. Singer, “A survey of structure from motion,” *Communications of the ACM*, 2017.
- [14] S. K. Prashanthi, S. Taluri, S. Beautlin, L. Karwa, and Y. Simmhan, “PowerTrain: Fast, generalizable time and power prediction models to optimize DNN training on accelerated edges,” *Future Generation Computer Systems*, vol. 161, pp. 329–344, 2024.
- [15] A. Spagnol, R. Le Riche, and S. Da Veiga, “Global sensitivity analysis for optimization with variable selection,” *SIAM/ASA Journal on Uncertainty Quantification*, vol. 7, no. 2, pp. 417–443, 2019.
- [16] G. J. Székely and M. L. Rizzo, “Brownian distance covariance,” *The Annals of Applied Statistics*, vol. 3, no. 4, pp. 1236–1265, 2009.

- [17] D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang, “Automatic database management system tuning through large-scale machine learning,” in *ACM International Conference on Management of Data (SIGMOD)*, 2017, pp. 1009–1024.
- [18] C. Wan, M. Santriaji, E. Rogers, H. Hoffmann, M. Maire, and S. Lu, “ALERT: Accurate learning for energy and timeliness,” in *USENIX Annual Technical Conference (ATC)*, 2020.
- [19] M. Yan, H. Wang, and S. Venkataraman, “PolyThrottle: Energy-efficient neural network inference on edge devices,” *arXiv preprint arXiv:2310.19991*, 2023.
- [20] J. You, J.-W. Chung, and M. Chowdhury, “Zeus: Understanding and optimizing GPU energy consumption of DNN training,” in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.
- [21] L. Zhang and E. Rupnik, “Sparsesat-nerf: Dense depth supervised neural radiance fields for sparse satellite images,” *arXiv preprint arXiv:2309.00277*, 2023.
- [22] Z. Zhang, Y. Zhao, H. Li, C. Lin, and J. Liu, “DVFO: Learning-based DVFS for energy-efficient edge-cloud collaborative inference,” *IEEE Transactions on Mobile Computing*, 2024.